GeoAlign-WAM: A 4D World Action Model for Generalizable and Robust Robot Manipulation Using a Video Foundation Prior

Zirui Li[1], Yujie Yan[1], Runjia Tan, Lei Yang, Yangang Ren, Ziwei Liu* and Chen Lv*

Author affiliations and mapping to be supplied,

ARTICLE INFO

Keywords:
Robot manipulation
World action models
Video foundation models
Pointmaps
Flow matching
Geometry–action alignment

ABSTRACT

Video foundation models provide temporal priors for learning robot manipulation policies, but predicting plausible image sequences does not explicitly constrain the relationship between visible scene geometry and Cartesian control. A pixel identifies a viewing ray rather than a unique metric location, whereas end-effector commands must resolve spatial relationships in the robot's workspace. We present GeoAlign-WAM, a geometry-aware world action model that combines a pretrained video prior with pixel-aligned pointmaps expressed in the robot base frame. RGB and pointmap sequences are encoded into compatible latent grids and fused along the spatial width dimension, enabling joint learning of appearance, three-dimensional structure, and their evolution over time. An action expert learns executable action chunks from the current multimodal context. To connect future world representations with control, a lightweight convolutional head predicts an auxiliary action chunk from the predicted future latent, and a symmetric geometry–action alignment loss penalizes disagreement with the action expert. Structured attention prevents the action branch from reading future tokens, allowing the future-generation branch and auxiliary head to be removed during deployment. The model achieves 97.4% success on LIBERO, 83.1% on LIBERO-Occ, and 84.0% across three real-robot tasks. Relative to Fast-WAM, the mean success rate increases by 11.9 percentage points under occlusion and 13.3 points on the real robot; LIBERO-Plus results show substantial gains under camera shifts, with remaining weaknesses under some language and lighting perturbations.

Project page: https://lzrbit.github.io/GeoAlign-WAM/.

1. Introduction

Robot manipulation requires a controller to translate an incomplete observation of a scene into physical interaction. A successful policy must identify task-relevant objects, estimate their relationship to the robot, and coordinate motion as contact and visibility change. Hand-designed perception and control pipelines can be effective in well-characterized settings, but their components and interfaces often require task-specific engineering. Learning-based approaches instead use experience to connect sensory observations with actions. End-to-end visuomotor learning demonstrated that perception and control can be optimized together, while subsequent imitation-learning methods improved the modeling of multimodal behavior and temporally extended action sequences [5, 13, 31]. These capabilities make learned policies attractive for manipulation across varied objects and configurations. Their reliability outside the training distribution, however, depends on what aspects of the scene their representations preserve.

Foundation models offer a route to broader representations than those learned from a limited set of robot demonstrations. Vision–language–action (VLA) models transfer visual and linguistic knowledge into instruction-conditioned control [1, 12, 33]. This transfer supports semantic task interpretation, but semantic recognition alone does not specify the metric displacement needed to approach a handle or place an object inside a receptacle. Video foundation models provide a complementary prior: they learn how appearance changes over time, including patterns associated with motion and interaction. World action models (WAMs) adapt such priors to robot control by coupling world prediction with action learning [28, 29]. The world objective can supervise properties of a demonstration that are not directly expressed by its action labels.

An important representational issue remains when the world objective is defined only through RGB observations or their latents. A three-dimensional point is projected to an image location after accounting for camera pose, perspective, and depth. Different metric configurations can therefore yield similar image evidence. Conversely, moving the camera

*Corresponding authors.
ziwei.liu@ntu.edu.sg (Z. Liu); lyuchen@ntu.edu.sg (C. Lv)
[1]Zirui Li and Yujie Yan contributed equally to this work.

can alter image coordinates without requiring the same change in a robot-frame action. Joint video and action training may learn these relationships implicitly, but the presence of a shared transformer does not itself establish an explicit, calibrated relationship between image locations and the robot workspace. We refer to this issue as a *geometric grounding gap*, rather than an inherent impossibility of learning spatial information from images.

This gap matters for generalization. A policy may perform well when object sizes, viewpoints, and backgrounds match its demonstrations, yet become sensitive to appearance changes that do not alter the required interaction. Partial occlusion further reduces the available evidence about object boundaries and approach directions. Explicit geometry provides an additional representation of visible spatial structure that can help disambiguate these cases. It cannot recover arbitrary hidden surfaces with certainty, nor does it make a policy automatically robust. The relevant question is whether geometry is both represented and used in a way that improves action selection.

Reliability under incomplete observations also depends on how an embodied agent acquires evidence and learns from experience. In object navigation, SOR-Nav decides when to continue local exploration or relocate to another region [9], while SafeVantage retains the viewpoints supporting a claim to guide further observation and selective decisions [14]. For manipulation, Predictive Action Chunk Learning (PACL) uses a predictive critic to learn from deployment rollouts of mixed quality [22]. These directions address evidence acquisition and policy improvement; our focus is the geometric representation linking current observations, predicted futures, and manipulation actions.

Recent work has already extended world action modeling toward spatial prediction. X-WAM incorporates future RGB-D modeling, and WAM4D uses spatial register tokens to transfer geometric supervision into action learning [7, 17]. Thus, adding a geometric output alone is not sufficient to define the contribution of a new model. We focus on a more specific connection: *the action implied by a predicted geometric world representation should agree with the action produced by the deployed policy*. This motivates a representation-level intervention and an action-level consistency objective within the same architecture.

We introduce GeoAlign-WAM, a 4D world action model with geometry–action alignment. Here, "4D" denotes a time-indexed sequence of three-dimensional scene representations. The model takes RGB observations together with pixel-aligned XYZ pointmaps in the robot base frame. A video-compatible encoding and fusion pathway exposes appearance and geometry to the world backbone, while an action expert predicts the control sequence. We draw on the multimodal representation strategy of 4DNeX [4] and the deployment structure of Fast-WAM [29], but use them to address action consistency in a metric workspace. In particular, a small convolutional readout converts the predicted future latent into an auxiliary action chunk. Its disagreement with the expert's action prediction defines a symmetric alignment loss, with gradients flowing through both branches.

The auxiliary readout is a training component. The action expert receives the current observation context and is prevented from attending to future world tokens. Consequently, deployment can omit future RGB/XYZ synthesis while retaining a representation shaped by future world prediction and action alignment. This design separates the information available to the policy from the supervision used to train it. The use of pointmaps introduces its own sensing and preprocessing cost, which must be included when measuring deployment latency.

The contributions of this work are as follows:

1. We formulate a pointmap-conditioned 4D WAM that brings metric scene geometry into the observation context and jointly learns future RGB, future XYZ, and robot actions. The representation preserves pixel correspondence while making its relationship to the robot base frame explicit.
2. We introduce a geometry–action alignment objective that compares an auxiliary action inferred from the predicted future latent with the action expert's prediction. The constraint operates on a common action representation and is compatible with a policy that does not consume future tokens at deployment.
3. We evaluate nominal performance, distribution shifts, and occlusion on LIBERO, LIBERO-Plus, and LIBERO-Occ [6, 16, 19], together with three real-robot tasks. The model reaches 97.4% on LIBERO and improves mean success over Fast-WAM by 11.9 percentage points on LIBERO-Occ and 13.3 points on the real robot.

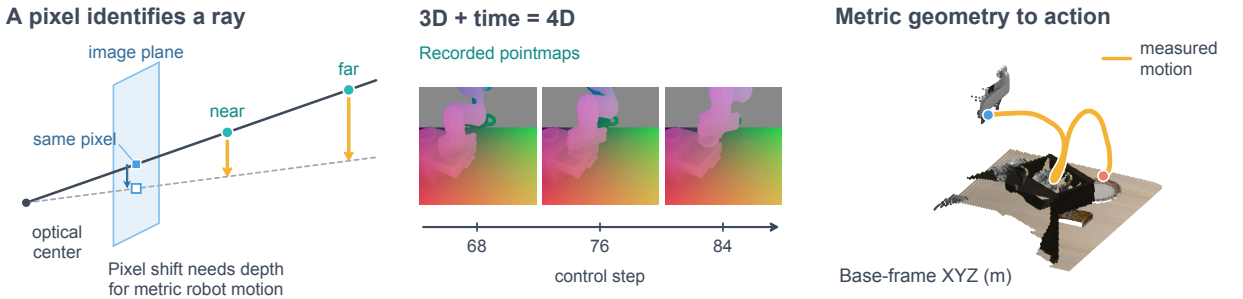


Figure 1: Pixel coordinates do not uniquely determine metric position. Robot-frame pointmaps provide 3D geometry; its evolution over time forms a 4D representation.

2. Related works

2.1. End-to-end visuomotor and geometry-aware manipulation

End-to-end manipulation is an established research direction: policies learn a mapping from sensory input to motor commands without requiring a separately specified symbolic planning interface. Levine et al. demonstrated joint learning of visual representations and control [13]. More recent imitation-learning policies model action distributions and action chunks. Diffusion Policy represents behavior through an iterative generative process, while Action Chunking with Transformers models temporally extended commands to support fine manipulation [5, 31]. These methods illustrate why a policy should account for both the ambiguity of individual actions and the temporal structure of demonstrations.

Geometry-aware policies address a different aspect of the learning problem: the representation of spatial relationships. PerAct uses a voxel-based representation for language-conditioned manipulation, and 3D Diffusion Policy combines a compact point-cloud representation with generative action modeling [23, 30]. These works establish that explicit 3D input is not new to manipulation. A related line of work aligns the internal features of VLA models with 3D representations; ROCKET, for example, applies residual multi-layer alignment to obtain spatially aware VLA features [24]. Our focus is the integration of pixel-aligned metric geometry into a video-derived world representation and its coupling to an action expert. The distinction concerns the training signal linking predicted scene evolution and actions, rather than the general use of depth or point clouds.

2.2. Vision–language–action models

VLA models combine visual perception, language conditioning, and robot action prediction in a common learning framework. RT-2 transfers knowledge from vision–language pretraining into robotic control, and OpenVLA provides an open model and adaptation framework for generalist manipulation [12, 33]. The π_0 model combines a pretrained vision–language backbone with flow-matching action generation [1], and $\pi_{0.5}$ extends it toward open-world generalization through co-training on heterogeneous data [21]. OpenVLA-OFT shows that the fine-tuning recipe, including parallel decoding and action chunking, strongly affects both speed and success [10]. Collectively, these approaches show how pretrained semantic representations can support control across tasks.

VLA and WAM are useful descriptions of model design, but they are not mutually exclusive categories. A language-conditioned policy can use predictive world objectives, and a WAM can expose the same observation-to-action interface as a VLA at deployment. We therefore distinguish prior work by the representation it learns and the information used during action inference. GeoAlign-WAM uses language to specify the task, RGB and XYZ to describe the current scene, and future world supervision to shape a video backbone. The proposed alignment objective addresses whether that future representation encodes action-relevant structure. It does not assume that VLA models are unable to infer geometry or dynamics.

2.3. World action models and 4D scene generation

Video-based robot models exploit visual prediction as a source of supervision for action learning. Unified World Models couples video and action diffusion within a shared transformer [32]. DreamZero is a representative large-scale WAM built around a video prior [28], and Cosmos Policy fine-tunes a pretrained video model for visuomotor control and planning [11]. Fast-WAM studies the role of future prediction during training and removes future-video generation from its deployment path [29]. We adopt the latter separation between current-context action inference and future-world

supervision. The additional question addressed here is how to make the learned world representation geometrically grounded and explicitly consistent with action prediction.

4DNeX models paired RGB and XYZ sequences in video-compatible latent space [4]. Its modality fusion provides a useful starting point, but generating dynamic scene geometry and controlling a calibrated robot impose different requirements. A robot policy must preserve the coordinate system and metric scale relevant to its controller, and its geometric representation must support executable commands. Our adaptation introduces observed robot-frame pointmaps into the conditioning context and connects predicted future features to the action space through an auxiliary readout. We credit the inherited fusion strategy rather than presenting concatenation as a new operation.

The closest spatial WAMs further clarify the scope of this contribution. X-WAM jointly models visual, geometric, and robot information using an RGB-D formulation with asynchronous denoising [7]. WAM4D introduces geometric supervision through spatial registers and removes that branch for efficient policy inference [17]. Our design uses explicit current XYZ input and a future-latent-to-action consistency loss within a video-derived model. Ablations are needed to isolate the effects of pointmap conditioning, future XYZ prediction, and alignment.

3. Problem formulation

3.1. Observations, world sequences, and action chunks

Consider an imitation-learning dataset $\mathcal{D}$ of synchronized robot demonstrations. At environment time t, the policy receives a language instruction ℓ and an observation

$$o_t = \left\{ I_t^{(v)}, P_t^{(v)} \right\}_{v=1}^{V}, \qquad I_t^{(v)} \in \mathbb{R}^{H_I \times W_I \times 3}, \quad P_t^{(v)} \in \mathbb{R}^{H_I \times W_I \times 3}. \tag{1}$$

Here v indexes the camera view and $P_t^{(v)}(u)$ gives the metric XYZ coordinate associated with image pixel u, expressed in the robot base frame B. RGB and XYZ must share a registered image grid. If a robot state is included by the chosen Fast-WAM configuration, it is treated as an additional current observation; no future state is available to the deployed policy. We suppress the view index below when the same operation applies to each view.

The action chunk is $A_t = [a_t, \dots, a_{t+H_A-1}] \in \mathbb{R}^{H_A \times d_a}$. We retain the action representation of the Fast-WAM task interface rather than replacing the controller with a new trajectory format. In particular, Cartesian action components must be distinguished from absolute end-effector positions. Pose increments, rotations, and gripper commands cannot all be interpreted as XYZ points. For the LIBERO interface, $d_a = 7$ and

$$a_t = [(u_t^p)^\top, (u_t^r)^\top, g_t]^\top, \qquad u_t^p, u_t^r \in \mathbb{R}^3, \tag{2}$$

where u_t^p is the translational increment command, u_t^r is the axis-angle rotation increment command, and g_t controls the gripper. Controller scaling converts these commands into pose offsets; their numerical values are not automatically displacements in meters. Any controller-frame to base-frame transformation must be consistent with the calibrated pointmaps. A normalization operator $\mathcal{N}_A$ maps the controller commands to the action expert's training space, and its inverse is applied before execution.

The future world target is

$$W_t = \left\{ I_{t+\delta_k}, P_{t+\delta_k} \right\}_{k=1}^{T}, \qquad 0 < \delta_1 < \cdots < \delta_T, \tag{3}$$

where the offsets δ_k align video sampling with the action interval. The video horizon T need not equal the action horizon H_A. The learned policy is

$$A_t \sim \pi_\theta(\cdot \mid o_t, \ell), \tag{4}$$

and only a prefix of the chunk is executed before observing the scene again. World prediction is an auxiliary conditional task trained on the same demonstrations. Under the attention structure introduced below, the world branch does not receive a candidate action sequence as input. It is consequently an observation-and-instruction-conditioned future predictor trained on demonstrations, not a counterfactual simulator for arbitrary interventions.

We use "4D representation" to refer to the sequence of 3D pointmaps in Eq. (3). Each map describes visible surfaces in a common coordinate frame. Pixel alignment between RGB and XYZ at one time does not imply persistent point identities across different times, complete reconstruction of occluded surfaces, or an explicit physical dynamics model.

3.2. Conditional flow-matching training

We formulate future RGB, future XYZ, and action prediction using conditional flow matching [18]. Let Y_m denote a normalized clean target for modality $m \in \{\text{rgb}, \text{xyz}, \text{act}\}$. For the visual modalities, Y_m is an encoded latent sequence; for actions, $Y_{\text{act}} = \mathcal{N}_A(A_t)$. For Gaussian noise $\epsilon_m \sim \mathcal{N}(0, \mathbf{I})$ and flow time $\tau_m \in (0, 1)$, define

$$Y_{m,\tau_m} = (1 - \tau_m)Y_m + \tau_m \epsilon_m, \qquad U_m = \epsilon_m - Y_m. \tag{5}$$

Thus $\tau_m = 0$ is clean data and $\tau_m = 1$ is noise. Environment time t, future-frame index k, and flow time τ_m are distinct quantities. With a velocity predictor $V_{\theta,m}$ and current context C_t, the modality objective is

$$\mathcal{L}^m_{\text{FM}} = \mathbb{E}\left[\frac{\omega_m(\tau_m)}{n_m}\left\|V_{\theta,m}(Y_{m,\tau_m}, \tau_m; C_t, \ell) - U_m\right\|_F^2\right], \tag{6}$$

where the norm is evaluated over valid, non-padded target entries, n_m is their count, and $\omega_m(\tau_m)$ denotes any baseline timestep weight (one for the unweighted objective). For RGB and XYZ, the predictor jointly reads both noisy latent grids through their fusion; the notation in Eq. (6) abbreviates this joint input. Current observation tokens remain clean and are excluded from the future-target loss.

We use a common world noise level τ_w for paired RGB and XYZ and denote the action noise level by τ_a. The distribution of these times and any noise-shift parameters are training settings, not additional contributions of the proposed alignment. A one-step estimate of the clean target is

$$\hat{Y}_m = Y_{m,\tau_m} - \tau_m \hat{U}_m, \qquad \hat{U}_m = V_{\theta,m}(\cdot). \tag{7}$$

Equation (7) is exact if the velocity prediction equals U_m; otherwise it is a one-step clean estimate. This estimate provides a common domain in which to compare the expert's action with the auxiliary prediction. A flow velocity is not itself an executable action.

Author check. The equations use Fast-WAM's clean-to-noise sign convention. Confirm the implemented scheduler, coupling of τ_w and τ_a, temporal offsets, action normalization statistics, and exact latent supplied to the convolutional neural network (CNN) head. The clean-latent readout in Eq. (7) is the explicit formulation used in this draft; an intermediate-feature implementation requires a corresponding revision.

4. Proposed model

4.1. Overall architecture

GeoAlign-WAM contains a world backbone initialized from a video foundation model, a modality-compatible RGB/XYZ encoding pathway, an action expert, and an auxiliary convolutional action head. Figure 2 separates the components required for deployment from those used for training. The observation encoders produce a clean multimodal context C_t. During training, the world backbone also processes noisy future RGB and XYZ latents and predicts their flow velocities. The action expert predicts action velocity using the current context and noisy actions. The auxiliary head reads the estimated clean future latent and outputs an action chunk in the same normalized space as the expert.

The design assigns different roles to the two action pathways. The expert is the policy used for control and can access only information available at the decision time. The auxiliary head measures whether a predicted future representation supports an action interpretation consistent with that policy. It provides a differentiable connection between the training objectives, but does not supply features to the action expert in the forward pass. Its output is neither averaged with the expert's action nor used as a second controller at deployment.

The pretrained world backbone and action expert are organized as a mixture of transformers: each branch has modality-specific transformations, while attention exposes the permitted context across branches. We retain the Wan2.2-5B backbone and action-expert configuration of Fast-WAM (Table 1). The addition of pointmap conditioning and alignment is intended to be evaluated without attributing improvements from a larger backbone or additional robot demonstrations to the proposed objective.

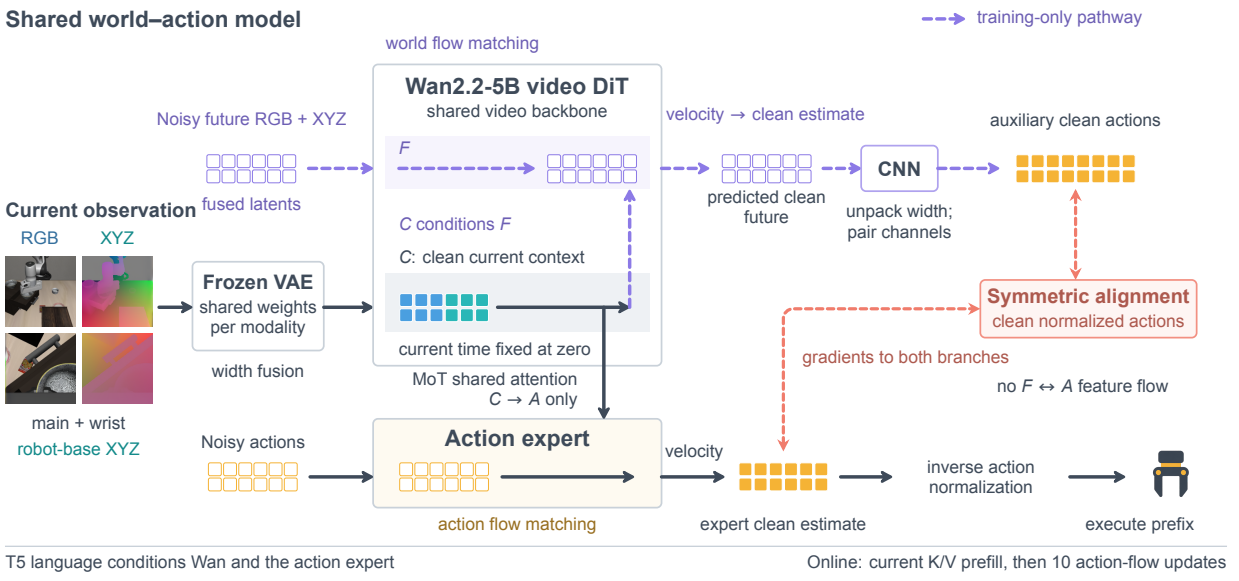


Figure 2: GeoAlign-WAM architecture. The Wan video backbone denoises future RGB/XYZ latents; an auxiliary readout aligns their action prediction with the action expert. Deployment uses only the current-context action path.

4.2. Input initialization and normalization

4.2.1. Metric pointmaps and registration

The geometric input is an observation in the robot base frame, rather than an uncalibrated color visualization of depth. For a depth-derived pointmap, the relevant coordinate transformation is

$$\begin{bmatrix} P_t^B(u) \\ 1 \end{bmatrix} = \mathbf{T}_{C,t}^B \begin{bmatrix} D_t(u)\mathbf{K}^{-1}\widetilde{u} \\ 1 \end{bmatrix}, \qquad \widetilde{u} = (u_x, u_y, 1)^\top, \tag{8}$$

where D_t is metric depth, $\mathbf{K}$ is the camera intrinsic matrix, and $\mathbf{T}_{C,t}^B$ maps camera coordinates to the robot base. This equation specifies the required geometry when depth is the source; it does not prescribe the pointmap acquisition method. A learned geometry estimator must resolve scale and frame alignment before its output is interpreted as a metric robot-frame pointmap. For a wrist camera, the extrinsic transform varies with the robot configuration.

Cropping, resizing, and view composition must be identical for each RGB/XYZ pair so that pixel correspondence is preserved. Views are packed in a fixed order using the baseline's camera layout, with the same layout applied to pointmaps. A pointmap pixel represents a scene surface point, not the pose or displacement of the end effector. Using a common base frame makes spatial relationships interpretable but does not imply direct equality between those quantities.

4.2.2. Initialization and metric normalization

The current XYZ input is initialized with the observed pointmap. A synthetic sloped-plane initialization, useful when geometry is unavailable in image-to-4D generation, would discard the measured spatial information in this setting. The known-context mask therefore treats valid current RGB and XYZ observations as conditioning data. Future RGB and XYZ targets are initialized from noise for generation. Reliability handling for missing or invalid depth is a separate preprocessing choice and must be reproduced in training and deployment.

To expose XYZ to a video-compatible encoder, let

$$\overline{P} = (P^B - \boldsymbol{b}) \oslash \boldsymbol{s}, \qquad P^B = \overline{P} \odot \boldsymbol{s} + \boldsymbol{b}, \tag{9}$$

where $\boldsymbol{b} \in \mathbb{R}^3$ and $\boldsymbol{s} \in \mathbb{R}^3_{>0}$ are known offset and scale vectors. The normalization must retain the information needed to recover the metric frame. A fixed transformation derived from training data is one suitable implementation. Independent centering of every frame would remove motion and location information unless the corresponding offsets were retained and provided to the model. Statistics computed from held-out trajectories or unavailable future frames must not be used to normalize the online observation.

RGB follows the input scaling of the pretrained encoder. XYZ additionally receives modality-specific normalization after encoding, as described next. Action normalization uses the baseline's controller-specific statistics and is shared by both action heads. The released Fast-WAM LIBERO configuration uses a per-component affine min/max mapping based on training statistics, with nominal range $[-1, 1]$; other robot interfaces require their own controller conventions. These three normalizations serve different purposes; latent normalization is not a replacement for camera calibration or action-unit conversion.

In our implementation, $\boldsymbol{b}$ is a fixed workspace center and $\boldsymbol{s} = s\mathbf{1}$ an isotropic scale; for the LIBERO experiments $\boldsymbol{b} = (0.531, 0.063, 0.214)\,\mathrm{m}$ and $s = 0.5793\,\mathrm{m}$. Normalized coordinates are clipped to $[-1, 1]$, and pixels without valid geometry are set to zero, which is the mid-gray value of the 8-bit visualization used in our figures. In simulation, pointmaps are back-projected from rendered metric depth with the simulator's camera intrinsics and camera-to-base transforms; points farther than 2 m from the camera and points more than 5 cm below the table surface are marked invalid. On the real robot, pointmaps are computed from the RGB-D depth images with the calibrated camera poses (Section 6).

Author check. Confirm that the real-robot policy uses the same $\boldsymbol{b}$ and s as the LIBERO model. The exported real-robot pointmap videos are consistent with these constants, but the deployed normalization file was not available to us.

4.3. Multimodal encoding

Let $\mathcal{E}_v$ and $\mathcal{D}_v$ be the encoder and decoder of the pretrained video variational autoencoder (VAE). Following the video-compatible encoding principle of 4DNeX, RGB and normalized pointmaps are encoded in separate applications of the VAE [4]. We write $\mathcal{E}_{\mathrm{rgb}}$ and $\mathcal{E}_{\mathrm{xyz}}$ to distinguish the modality pathways; these symbols do not require separately pretrained encoder weights. With frozen VAE parameters, their latent sequences are

$$Z^{\mathrm{rgb}} = \mathcal{E}_{\mathrm{rgb}}(\overline{I}), \qquad Z^{\mathrm{xyz}} = \mathcal{E}_{\mathrm{xyz}}(\overline{P}), \qquad Z^{m} \in \mathbb{R}^{T_z \times h \times w \times c}. \tag{10}$$

Here T_z, h, w are the compressed temporal and spatial sizes, and c is the latent channel count. The pointmap branch operates on an organized XYZ image, not an unordered point cloud. Its use of a video VAE is a representational choice that makes the two modalities compatible with the backbone.

For XYZ latent statistics $\boldsymbol{\mu}_{\mathrm{xyz}}$ and $\boldsymbol{\sigma}_{\mathrm{xyz}}$ estimated on the training set, define

$$\widetilde{Z}^{\mathrm{xyz}} = (Z^{\mathrm{xyz}} - \boldsymbol{\mu}_{\mathrm{xyz}}) \oslash (\boldsymbol{\sigma}_{\mathrm{xyz}} + \varepsilon). \tag{11}$$

Broadcasting is over the corresponding latent channels. Before decoding, this transformation is inverted and Eq. (9) restores robot-frame units. The RGB latent follows the pretrained backbone's scaling. Throughout the remaining formulation, both modality latents denote their normalized versions unless stated otherwise.

The current context must be encodable without access to future frames. We therefore use the causal observation encoding supported by the baseline or encode the current observation separately. Encoding a full demonstration with a temporally non-causal encoder and then selecting its first token would violate this requirement. The same current-context construction is used during training and inference, with its diffusion-time embedding fixed at the clean endpoint $\tau_C = 0$. In particular, context modulation must not change with the sampled future or action noise level.

Language embeddings condition both branches through the baseline's text-conditioning pathway. Visual tokens retain temporal and spatial position information. The 4DNeX-style encoding considered here adds distinct modality embeddings and shares underlying time and image-grid indices across paired RGB/XYZ tokens. This separates pixel correspondence from the bookkeeping position created by concatenation. The precise positional-encoding choice is retained as an implementation check, since width concatenation alone does not determine it.

4.4. Fusion strategies

The final model uses width-wise concatenation of RGB and XYZ latents:

$$Z^{F} = \mathcal{F}_w(Z^{\mathrm{rgb}}, Z^{\mathrm{xyz}}) = \mathrm{Concat}_w(Z^{\mathrm{rgb}}, Z^{\mathrm{xyz}}) \in \mathbb{R}^{T_z \times h \times 2w \times c}. \tag{12}$$

The same operation is applied to the current context and future targets. In our implementation, each latent sequence Z^m already packs the main and wrist cameras side by side, following the two-camera layout of the baseline, and each camera panel is 224×224 pixels. The fused model frame therefore arranges four panels in the order RGB main, RGB wrist, XYZ main, XYZ wrist, giving a 224×896 frame (Fig. 3). This preserves the pretrained latent channel width,

while allowing attention to exchange information between modalities. The packed width is an architectural layout; the XYZ half is not a second camera view and does not represent a different physical pixel location merely because it is placed to the right of RGB. Whether paired tokens share rotary indices or use explicit packed positions is a separate encoding choice.

To isolate the effect of this choice, the fusion study considers the following alternative constructions inspired by 4DNeX's modality-fusion analysis [4]:

- Channel fusion: concatenate at matching locations along the feature-channel dimension and adapt the input/output projections to the increased channel count.
- Batch fusion: process modality-labeled samples separately. Without an explicit cross-modal interaction mechanism, batch concatenation alone does not fuse information within a sample.
- Frame fusion: pack modality sequences along the temporal axis, retaining separate modality labels and the original physical timestamps.
- Height fusion: place the latent grids above and below one another while retaining their shared image coordinates.
- Width fusion: use the side-by-side layout of Eq. (12), selected for the final model.

Figure 3 summarizes these layouts, and Appendix A gives their tensor shapes, index mappings, tokenization, and cross-modal interaction formulas. Channel fusion changes projection dimensions; other strategies may change token count or attention cost. A fair comparison must therefore report these differences alongside task performance. We do not interpret concatenation alone as a geometric constraint. Its role is to expose paired modalities to shared computation; the proposed action-consistency objective supplies an additional task-level connection.

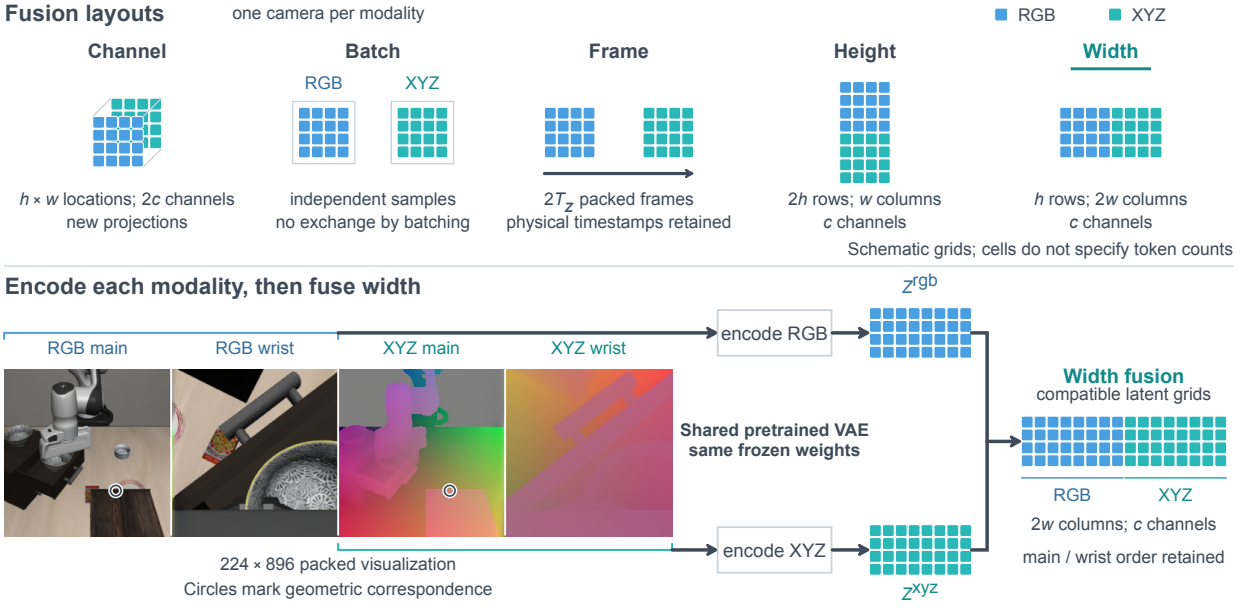


Figure 3: RGB–pointmap fusion layouts and the selected width-wise packing of two camera views. Paired RGB and XYZ retain pixel correspondence; Appendix A gives the formulations.

4.5. Causal world–action transformer

Let C denote clean current RGB/XYZ context tokens, F noisy future fused tokens, and A noisy action tokens. We extend the Fast-WAM visibility structure to the multimodal context [29], as illustrated in Fig. 4. For a query-group/key-group ordering (C, F, A), the allowed-attention matrix is

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}. \tag{13}$$

An entry of one permits all attention edges within the corresponding block. The additive attention mask $\mathbf{M}$ is zero on permitted edges and $-\infty$ elsewhere, giving

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d}} + \mathbf{M}\right) V. \tag{14}$$

RGB and XYZ may interact within C and within F. The action group reads C and its own chunk, while the future group reads C and future world tokens. Language conditioning is available separately to the permitted branches.

Causality here concerns the separation of available observations from unavailable future information. It does not mean that every future video frame or every action token is processed by a strictly autoregressive temporal mask. Bidirectional attention within a chunk is compatible with generating that chunk jointly. Crucially, the context cannot read future or action tokens, so it cannot relay future information indirectly into the action expert through additional transformer layers. Attention masking must be accompanied by causal preprocessing and conditioning: normalization, temporal encoding, and any pooled features must obey the same visibility constraint.

This structure also determines the interpretation of the model. World and action branches share context and training supervision, but do not implement a general action-conditioned transition distribution. Their outputs are tied through the alignment objective in the next subsection. The mask ensures that the policy's forward computation is unchanged when the future branch is removed.

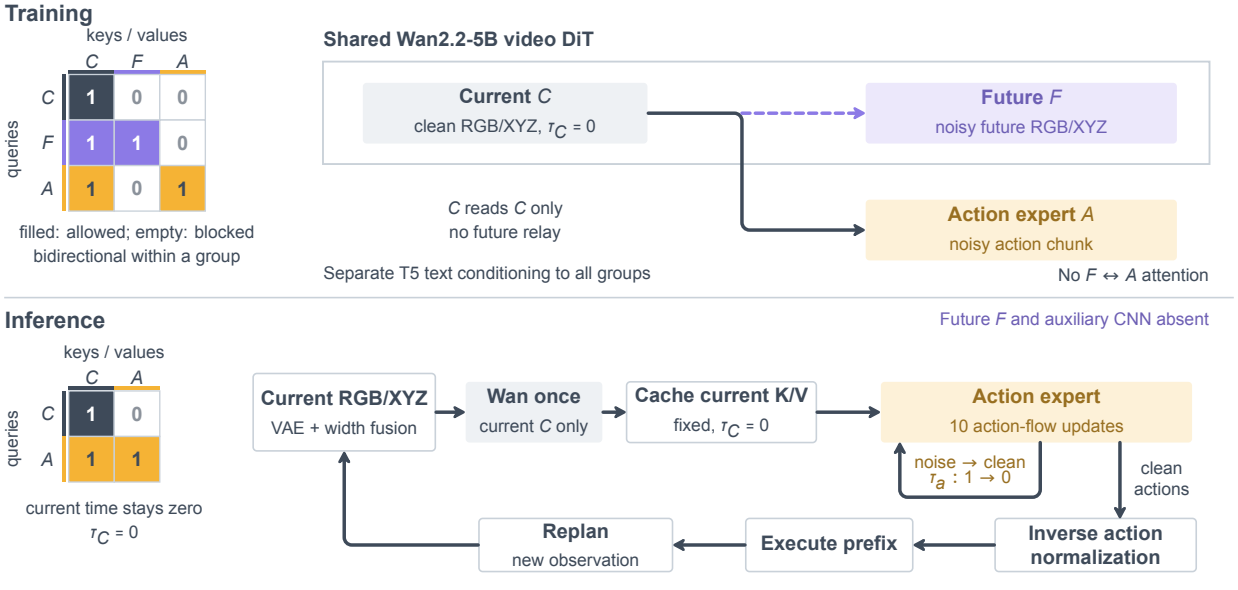


Figure 4: Training and inference attention. Training separates future-world and action tokens; inference removes future tokens and reuses cached current-context keys and values for action denoising.

4.6. Geometry–action alignment and training objective

4.6.1. Auxiliary action from the predicted future latent

Using Eq. (7), reconstruct the normalized clean future estimates $\widehat{Z}^{\mathrm{rgb}}$ and $\widehat{Z}^{\mathrm{xyz}}$. Before applying local convolutions, undo the side-by-side width packing and align the two feature grids by pixel position:

$$\widehat{G} = \mathrm{Concat}_c(\widehat{Z}^{\mathrm{rgb}}, \widehat{Z}^{\mathrm{xyz}}), \qquad \widehat{G} \in \mathbb{R}^{T_z \times h \times w \times 2c}. \tag{15}$$

This readout layout differs from the transformer's width layout. It avoids treating the artificial boundary between modality panels as a physical image neighborhood. A small convolutional head $\mathcal{H}_\psi$ maps the aligned future features to the action horizon:

$$\widehat{A}^{\mathrm{aux}} = \mathcal{H}_\psi(\widehat{G}) \in \mathbb{R}^{H_A \times d_a}. \tag{16}$$

Spatial feature extraction and temporal projection allow T_z and H_A to differ. The head is a readout of predicted world features; it does not consume the action expert's hidden tokens or ground-truth actions as input. Its exact convolutional layout is an implementation parameter, rather than an assumption that one XYZ pixel corresponds to one robot command.

The expert's clean action estimate under the same flow convention is

$$\widehat{A}^{\mathrm{exp}} = A_{\tau_a} - \tau_a V_{\theta,\mathrm{act}}(A_{\tau_a}, \tau_a; C_t, \ell), \tag{17}$$

where A_{τ_a} denotes the noisy normalized action chunk. Both estimates use identical action units, channel ordering, horizon, and normalization. Comparing Eq. (16) directly with an action velocity would instead mix incompatible targets.

4.6.2. Symmetric alignment

The geometry–action alignment objective is

$$\mathcal{L}_{\mathrm{align}} = \mathbb{E}\left[\frac{1}{H_A d_a}\left\|\widehat{A}^{\mathrm{aux}} - \widehat{A}^{\mathrm{exp}}\right\|_F^2\right]. \tag{18}$$

There is no stop-gradient operation on either prediction. Writing $\Delta = \widehat{A}^{\mathrm{aux}} - \widehat{A}^{\mathrm{exp}}$, the two output gradients are proportional to Δ and $-\Delta$, respectively. Thus the loss updates both the future-world readout pathway and the action expert, in addition to any shared trainable parameters. All comparisons are made between predictions associated with the same demonstrated interval.

This objective encourages agreement in action space; it is not a point-to-end-effector distance, a camera-calibration loss, or a proof of physical consistency. A flexible CNN can in principle exploit appearance features without using XYZ. Likewise, two incorrect action predictions can agree. The value of the term therefore depends on its interaction with supervised world/action learning and must be established by the alignment and geometry ablations. If the demonstration distribution contains several valid futures, independently sampled world and action trajectories need not represent the same mode. During training, Eq. (18) uses paired demonstration targets and their noisy interpolants. It should not be interpreted as guaranteeing consistency between arbitrary independently sampled rollouts.

4.6.3. Joint objective

The complete training loss is

$$\mathcal{L} = \lambda_a \mathcal{L}_{\mathrm{FM}}^{\mathrm{act}} + \lambda_r \mathcal{L}_{\mathrm{FM}}^{\mathrm{rgb}} + \lambda_p \mathcal{L}_{\mathrm{FM}}^{\mathrm{xyz}} + \lambda_{\mathrm{align}} \mathcal{L}_{\mathrm{align}}, \tag{19}$$

where the nonnegative weights control the balance of modalities and consistency. The RGB and XYZ objectives are latent velocity losses. They anchor the world branch to demonstrated appearance and geometry, while the action objective anchors the expert to demonstrated commands. We do not add an unreported auxiliary imitation loss or an extra geometric reprojection objective. In particular, latent XYZ flow matching is distinct from the 4DNeX-style post-optimization that we use for offline reconstruction of predicted future pointmaps (Appendix B); that fitting procedure does not update the policy or add a training loss.

The alignment gradient enters both a high-dimensional future predictor and a comparatively compact action predictor, so its scale can affect optimization. Reporting the weight and the separately normalized component losses is necessary

to interpret the method. A zero-alignment ablation retains pointmap input and future XYZ learning; a zero-XYZ-loss ablation tests a different mechanism and should not be conflated with removing pointmap observations.

Author check. Confirm whether the implemented CNN uses the rearrangement in Eq. (15) or the raw fused layout. The rearrangement is a proposed explicit realization of the readout. Supply the CNN layers, latent tap, output projection, loss weights, modality/position embeddings, and trainable parameter groups; no values have been invented here.

4.7. Efficient inference

At deployment, the model receives current RGB and robot-frame pointmaps, constructs their fused clean context, and encodes that context through the world backbone once. Because C is independent of F and A in Eq. (13), its attention keys and values can be cached for the subsequent action steps. The action expert begins from $A^{(K)} \sim \mathcal{N}(0, \mathbf{I})$ and follows a decreasing flow-time schedule $1 = \tau_K > \cdots > \tau_0 = 0$. A first-order update illustrating the sign convention is

$$A^{(k-1)} = A^{(k)} + (\tau_{k-1} - \tau_k) V_{\theta,\mathrm{act}}(A^{(k)}, \tau_k; C_t, \ell). \tag{20}$$

Following Fast-WAM, we use $K = 10$ Euler updates and compute the current-context keys and values once at $\tau_C = 0$. The resulting normalized chunk is converted back to controller commands using $\mathcal{N}_A^{-1}$. The robot executes a prefix and replans with a new observation.

Future RGB and XYZ tokens, future latent decoding, and the auxiliary CNN are omitted from the online action path. This removes their deployment cost, while preserving current pointmap acquisition, encoding, and multimodal context processing. The method therefore avoids future-world denoising at test time; it does not imply one-step action generation or zero geometric preprocessing cost. A suitable latency decomposition is

$$t_{\mathrm{policy}} = t_{\mathrm{pointmap}} + t_{\mathrm{encode}} + t_{\mathrm{context}} + K\, t_{\mathrm{action\ step}} + t_{\mathrm{post}}, \tag{21}$$

with sensing and communication accounted for consistently across compared systems. This expression is a measurement convention rather than a claimed speedup.

The intended deployment advantage is that future geometric supervision can influence learning without requiring future geometric synthesis for each decision. Whether the resulting representation improves generalization or occlusion robustness is an empirical question. Metric errors, calibration drift, and severe missing geometry can still degrade the policy, and the present objective alone does not guarantee recovery from those conditions.

5. Simulation experiments

The simulation study uses three benchmarks that share the LIBERO task interface but change different factors at test time. LIBERO measures nominal task performance, LIBERO-Plus applies controlled distribution shifts to the same tasks, and LIBERO-Occ hides task-relevant objects from the third-person camera. All three are evaluated with the same trained policy and the same inference settings.

5.1. Benchmarks and protocol

LIBERO [19] contains four task suites, Spatial, Object, Goal and Long, with ten language-conditioned tasks each, performed by a simulated Franka arm. For our evaluations, each task is tested over 100 episodes using the benchmark initial states, giving 1,000 episodes per suite. Success rates are reported by suite and across the four suites.

LIBERO-Plus [6] perturbs the LIBERO suites along seven dimensions: camera viewpoint, robot initial state, language instruction, lighting, background texture, sensor noise and object layout. Its 10,030 task variants are evaluated with policies trained on the original LIBERO demonstrations, without retraining. We report results by suite and perturbation dimension together with the unperturbed LIBERO result of each method (Table 3).

LIBERO-Occ [16] provides occluded counterparts of the four suites. Occluders are LIBERO objects placed along the line of sight from the third-person camera to the task-relevant object, and a placement is retained only if it hides the target in that view, does not collide with the scene and still allows the original demonstration to be replayed. Each suite contains ten tasks. We evaluate each task over 100 episodes using the benchmark initial states and report the mean success rate over the 1,000 episodes in each suite.

Author check. State how the LIBERO-Plus sensor-noise perturbation enters the inputs of GeoAlign-WAM. The perturbation is defined on the rendered camera image, whereas our pointmaps are rendered directly from simulator depth; the Noise column is comparable with the RGB-only baselines only if the corrupted RGB is used and the treatment of the pointmap is reported.

Table 1
Model and experiment configuration. Backbone and action-expert settings follow Fast-WAM [29]; unresolved training settings remain marked.

Parameter	Setting
Video foundation model	Wan2.2-TI2V-5B
World transformer	30 layers; hidden width 3072; feed-forward width 14336
Action transformer	30 layers; hidden width 1024; feed-forward width 4096
Attention configuration	24 heads, head dimension 128 in both branches
Video latent / patch size	48 channels / $1 \times 2 \times 2$
Text conditioning	T5 features of width 4096; up to 128 tokens
Model size	6.3B parameters
Camera inputs	Main and wrist RGB-D; XYZ expressed in the robot base frame
Panel / packed frame	224×224 / 224×896 pixels
Visual training sequence	9 frames including the observation; 4 control steps apart
Action / proprioception	7 / 8 dimensions in the LIBERO interface
Action chunk / execution	32 predicted actions / first 10 executed before replanning
Action inference	10 Euler updates; current-context keys and values cached once
Control frequency	20 Hz
LIBERO / LIBERO-Occ evaluation	100 episodes per task; 10 tasks per suite
Real-robot evaluation	25 trials per reported policy–task pair

5.2. Baselines

The simulation comparisons cover three groups:

- Vision–language–action models: OpenVLA [12], OpenVLA-OFT [10], RIPT-VLA [25], π_0 [1], π_0-FAST [20], $\pi_{0.5}$ [21], UniVLA [2], and NORA [8]. The LIBERO-Occ comparison additionally includes the Unified Vision-Language-Action Model [27], also named UniVLA but distinct from Bu et al. [2].
- World action models: Cosmos Policy [11], Fast-WAM [29], and WorldVLA [3]. Fast-WAM is the direct architectural reference: GeoAlign-WAM retains its video backbone, action interface, and deployment procedure while adding pointmap input, future XYZ supervision, and action alignment.
- Geometry-aware policies: $\pi_{0.5}$ with Spatial Forcing [15] or ROCKET alignment [24].

For LIBERO-Occ, Table 4 distinguishes main-camera-only policies from the main-and-wrist-camera inputs of Fast-WAM and GeoAlign-WAM.

5.3. Implementation details

GeoAlign-WAM is implemented on top of Fast-WAM [29], whose world backbone is the video diffusion transformer of Wan2.2-5B [26] and whose action expert is a smaller transformer coupled to the backbone as a mixture of transformers with shared attention. The policy observes the third-person (agentview) camera and the wrist camera. For each camera, the simulator renders RGB and metric depth, and the depth is converted into a base-frame pointmap with Eq. (8) and normalized as described in Section 4.2. The four 224×224 panels are packed into the 224×896 model frame of Section 4.4. The video target contains the current frame and eight future frames spaced four control steps apart, so that the predicted world covers the 32-step action chunk. Actions use the seven-dimensional delta end-effector interface of Eq. (2) at a control rate of 20 Hz. At test time, the action expert integrates the flow with $K = 10$ steps, the first ten actions of each chunk are executed, and the policy replans from the new observation. Each episode starts after 30 settling steps and is limited to 400 control steps; evaluation uses a fixed random seed. Table 1 summarizes the model and evaluation configuration.

Author check. Confirm the optimizer, effective batch size, training duration, low-rank adaptation (LoRA) configuration, precision, hardware, loss weights, and CNN architecture. Run names indicate a learning rate of 10^{-4} and a case-study checkpoint at step 9,500, but do not establish the full training configuration. Also confirm whether the 400-step evaluation limit applies to every suite; Fast-WAM's public evaluator uses 700 steps for LIBERO-10.

5.3.1. Case-study visualization

The case-study figures are generated from exported rollouts using Python and Matplotlib. RGB and pointmap frames are selected by timestamp; joint, gripper, and 3D end-effector plots use recorded arrays. The real-robot joint reconstruction is described in Section 6.5. Numerical values in the case descriptions are extracted from the corresponding recordings.

5.4. Results

5.4.1. Nominal task performance

Table 2 summarizes LIBERO and LIBERO-Plus performance. On nominal LIBERO, GeoAlign-WAM achieves 97.4% mean success, with suite rates of 98.1%, 98.3%, 96.9%, and 96.3% for Spatial, Object, Goal, and Long, respectively (Table 3). Its overall rate is 0.2 percentage points below Fast-WAM's 97.6%. Thus the principal observed benefit concerns performance under visual shifts and occlusion rather than an increase on the nominal benchmark.

5.4.2. Robustness to distribution shifts

With equal weight assigned to the four suites and seven perturbation dimensions, GeoAlign-WAM achieves 81.0% mean success on LIBERO-Plus (Table 2). Table 3 shows that the difference from Fast-WAM depends on the perturbation. Under camera shifts, GeoAlign-WAM reaches 73.4%, 94.5%, 95.4%, and 54.3% on the four suites, improving over Fast-WAM by 59.0, 69.2, 87.3, and 36.6 percentage points. The gains extend to robot-state changes in Goal (89.3% versus 24.7%) and to background and sensor-noise changes in Long (87.4% versus 22.8%, and 83.2% versus 28.5%). These results show that the nominal success rate alone does not capture robustness to altered observations.

The improvement is not uniform. In LIBERO-Object, layout perturbations yield 72.8%, slightly below Fast-WAM's 73.0%; Goal lighting falls to 63.1% versus 75.3%; and Long language perturbations reach 58.2% versus 60.1%. Relative to the stronger VLA baselines, GeoAlign-WAM also remains weaker in several language and layout settings. For example, $\pi_{0.5}$ reaches 89.6% under Long language changes and 98.2% under Spatial layout changes, compared with 58.2% and 81.8% for GeoAlign-WAM. The results therefore support gains in specific visual robustness settings, without establishing uniformly better generalization across perturbations.

5.4.3. Occlusion robustness

On LIBERO-Occ (Table 4), GeoAlign-WAM achieves 83.1% mean success compared with 71.2% for Fast-WAM, an increase of 11.9 percentage points. The gains are 9.9, 11.9, 12.0, and 13.9 points on Spatial, Goal, Object, and LIBERO-10, respectively. Both methods use main and wrist cameras, so this comparison holds camera availability fixed. The first five baselines use only the main view; their rates provide additional context but do not isolate model differences from access to the wrist view. The improvement over Fast-WAM occurs in all four suites, including the longer task sequences, while the case studies illustrate how the two observed views differ when one is occluded.

5.4.4. Ablation scope

The full-model comparisons do not separate the contributions of pointmap input, future XYZ supervision, fusion layout, and alignment. Component-level conclusions require controlled ablations with the same backbone, data, and evaluation protocol.

Draft status. Add the fusion-strategy, pointmap-input, XYZ-loss, and alignment-loss ablations when their results are available.

Table 2
Aggregate success rates (%) on LIBERO and LIBERO-Plus [6]. Orig. and Plus denote performance on original LIBERO and overall LIBERO-Plus, respectively; Size denotes model size. Cam.: camera viewpoint; Robot: robot initial state; Lang.: language instruction; Light: lighting; BG: background texture; Noise: sensor noise; Layout: object layout. For GeoAlign-WAM, each perturbation score is the equal-weight mean over four suites; Plus is the equal-weight mean over all 28 suite–perturbation entries.

Method	Size	Orig.	Plus	Cam.	Robot	Lang.	Light	BG	Noise	Layout
VLAs										
$\pi_{0.5}$ [21]	3.3B	96.9	84.6	72.0	76.6	86.5	96.1	95.2	86.7	86.0
OpenVLA-OFT [10]	7B	97.1	69.6	56.4	31.9	79.5	88.7	93.3	75.8	74.3
RIPT-VLA [25]	7B	97.5	68.4	55.2	31.2	77.6	88.4	91.6	73.5	74.2
π_0 [1]	3.3B	91.3	69.3	61.0	40.8	63.7	89.3	84.1	80.1	75.9
π_0-FAST [20]	3.3B	85.5	61.6	65.1	21.6	61.0	73.2	73.3	74.4	68.8
UniVLA [2]	8.5B	95.2	42.9	1.8	46.2	69.5	69.0	81.0	21.2	31.9
NORA [8]	3B	87.9	39.0	2.2	37.0	65.1	45.7	58.6	12.8	62.1
OpenVLA [12]	7B	76.5	15.6	0.8	3.5	23.0	8.1	34.8	15.2	28.5
WAMs										
Cosmos-Policy [11]	2B	98.5	82.4	73.4	63.3	89.3	98.9	83.5	89.3	84.0
Fast-WAM [29]	6B	97.6	50.0	16.4	44.5	68.9	78.2	53.7	37.7	60.7
WorldVLA [3]	7B	79.1	25.0	0.1	27.9	41.6	43.7	17.1	11.0	38.0
Geometry-aware VLAs										
$\pi_{0.5}$ + Spatial Forcing [15]	3.3B	94.0	25.7	0.1	0.3	26.8	66.0	45.9	0.1	59.8
$\pi_{0.5}$ + ROCKET [24]	3.3B	95.3	47.5	30.9	75.6	29.3	69.2	47.0	25.4	62.0
GeoAlign-WAM (ours)	6.3B	97.4	81.0	79.4	72.2	77.7	81.4	95.2	88.6	72.5

Table 3
LIBERO-Plus success rates (%) by task suite and perturbation. Orig. denotes unperturbed LIBERO. Cam.: camera viewpoint; Robot: robot initial state; Lang.: language instruction; Light: lighting; BG: background texture; Noise: sensor noise; Layout: object layout. π_0^* denotes the results reported by LIBERO-Plus [6] for the official openpi LIBERO checkpoint.

Method	Orig.	Cam.	Robot	Lang.	Light	BG	Noise	Layout
(a) LIBERO-Spatial								
GeoAlign-WAM (ours)	98.1	73.4	78.3	95.5	98.0	99.4	84.3	81.8
$\pi_{0.5}$ [21]	98.8	76.6	86.3	96.4	97.9	99.6	92.0	98.2
Cosmos Policy [11]	98.1	83.5	59.7	96.4	99.7	85.3	91.7	95.1
Fast-WAM [29]	98.2	14.4	44.0	69.5	87.3	69.8	35.0	60.5
OpenVLA-OFT [10]	97.6	88.3	40.0	80.5	98.3	97.3	96.3	93.9
RIPT-VLA [25]	97.5	85.4	38.0	99.7	99.7	100.0	92.0	92.3
π_0 [1]	96.8	70.7	49.1	67.9	92.8	95.0	87.7	94.0
π_0^* [6]	96.8	17.8	6.6	58.8	89.7	90.7	90.9	89.1
π_0-FAST [20]	96.4	87.2	26.9	84.2	37.0	97.7	93.2	95.5
UniVLA [2]	96.5	1.1	52.6	83.9	96.6	90.7	15.7	69.5
NORA [8]	92.2	4.3	50.9	63.8	66.8	65.5	12.5	84.6
WorldVLA [3]	85.6	0.0	44.3	46.3	65.1	19.8	11.7	46.1
OpenVLA [12]	84.7	0.0	3.7	27.7	12.3	50.4	12.0	40.7
$\pi_{0.5}$ + ROCKET [24]	96.4	14.9	18.0	35.4	50.7	45.0	13.7	48.6
(b) LIBERO-Object								
GeoAlign-WAM (ours)	98.3	94.5	71.8	98.8	100.0	95.8	97.3	72.8
$\pi_{0.5}$	98.2	86.4	71.9	91.0	99.0	99.2	96.2	91.1
Cosmos Policy	100.0	88.6	61.6	94.1	99.7	96.8	97.4	86.4
Fast-WAM	100.0	25.3	64.1	96.3	97.9	77.8	63.7	73.0
OpenVLA-OFT	98.4	38.9	25.4	99.0	73.7	97.6	72.3	71.8
RIPT-VLA	97.5	37.9	26.4	80.8	85.9	99.2	68.0	70.1
π_0	98.8	80.1	31.9	75.4	94.3	85.9	87.9	76.2
π_0^*	98.8	22.2	8.3	70.0	90.9	91.1	87.0	76.2
π_0-FAST	96.8	72.0	27.6	71.5	71.0	95.2	93.1	84.5
UniVLA	96.8	0.0	42.2	86.9	25.6	81.5	10.4	27.3
NORA	95.4	0.5	28.4	76.4	25.3	54.8	5.7	55.8
WorldVLA	89.0	0.0	26.4	57.2	20.5	17.3	18.0	53.6
OpenVLA	88.4	0.5	4.5	21.0	1.0	45.2	11.4	22.4
$\pi_{0.5}$ + ROCKET	98.8	41.4	27.1	44.6	91.6	69.8	34.1	83.6
(c) LIBERO-Goal								
GeoAlign-WAM (ours)	96.9	95.4	89.3	58.4	63.1	98.2	89.6	59.6
$\pi_{0.5}$	98.0	77.2	73.6	70.5	93.2	92.5	87.3	70.4
Cosmos Policy	98.2	64.0	64.1	73.4	100.0	85.1	75.2	65.2
Fast-WAM	97.0	8.1	24.7	49.8	75.3	44.5	23.7	48.0
OpenVLA-OFT	97.9	62.0	25.2	53.2	93.9	92.5	75.2	59.1
RIPT-VLA	97.5	65.7	23.2	45.4	74.2	79.7	71.0	59.8
π_0	95.8	56.6	43.3	43.2	90.3	84.7	82.8	59.8
π_0^*	95.8	12.3	5.6	39.3	84.2	76.5	76.5	44.7
π_0-FAST	88.6	70.8	20.5	47.3	95.3	60.9	69.7	51.6
UniVLA	95.6	3.9	37.9	45.6	89.6	78.3	33.5	22.6
NORA	89.4	2.9	31.1	56.6	60.6	60.5	18.2	53.9
WorldVLA	82.6	0.3	30.6	42.2	68.8	30.3	13.5	47.4
OpenVLA	79.2	2.5	2.7	21.5	9.0	27.1	19.5	25.6
$\pi_{0.5}$ + ROCKET	96.6	41.2	36.9	27.6	57.0	47.0	28.2	46.8
(d) LIBERO-Long								
GeoAlign-WAM (ours)	96.3	54.3	49.3	58.2	64.3	87.4	83.2	75.6
$\pi_{0.5}$	92.4	49.2	76.1	89.6	93.8	90.3	73.1	85.9
Cosmos Policy	97.6	58.9	67.4	94.8	96.4	68.9	91.8	92.9
Fast-WAM	95.2	17.7	45.3	60.1	52.2	22.8	28.5	61.2
OpenVLA-OFT	94.5	38.7	38.2	87.0	89.4	86.8	63.5	76.9
RIPT-VLA	97.5	34.1	38.4	88.3	93.4	89.3	66.4	79.2
π_0	73.8	38.7	39.9	69.7	79.2	72.3	64.6	77.6
π_0^*	85.2	3.8	3.6	68.4	74.5	69.5	64.4	69.6
π_0-FAST	60.2	33.2	12.0	43.6	91.6	44.6	46.1	47.8
UniVLA	92.0	1.9	53.2	64.2	65.7	74.4	25.4	16.4
NORA	74.6	1.2	39.4	64.0	30.3	54.0	15.1	59.5
WorldVLA	59.0	0.0	12.2	20.6	20.4	1.7	1.6	4.4
OpenVLA	53.7	0.0	3.0	22.2	10.6	19.4	17.6	28.3
$\pi_{0.5}$ + ROCKET	89.2	25.3	38.4	11.0	77.0	29.4	23.8	71.5

5.5. Case studies

Figures 5–7 follow three closed-loop rollouts of the same LIBERO-Spatial instruction, "pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate", under nominal conditions, a LIBERO-Plus camera shift and a LIBERO-Occ occluder. Each figure shows five keyframes of the main camera, the base-frame pointmaps of the same frames, the wrist RGB image and pointmap shortly before the grasp, the seven joint angles with the gripper opening and the replanning instants, and the 3D end-effector trajectory. The rollouts are successful episodes selected for illustration; the benchmark statistics are those of Tables 2–4. All images were rendered offline from the recorded simulator states.

5.5.1. *Case S1: nominal scene*

In the nominal case (Fig. 5), the end effector starts at (0.45, 0.02, 0.27) m in the base frame and descends into the open top drawer, where the bowl sits below the drawer rim. The gripper closes at step 60 ($t = 3.0$ s), and the measured finger opening falls from 80 mm to 5 mm as the rim is pinched. The bowl is then lifted to $z = 0.32$ m while the arm travels about 0.23 m along y_B toward the plate, and the success condition is reached at $t = 7.2$ s with the end effector at (0.72, 0.25, 0.03) m. The drawer interior has little texture in RGB but appears in the pointmap as a recessed region with its own coordinates, and the bowl changes color in the pointmap as its position changes. Shortly before the grasp ($t = 2.25$ s), the wrist camera looks down into the open drawer with the bowl between the fingers, and the drawer, its handle and the bowl stand out from the table in the wrist pointmap. The joint trajectories remain smooth across all 15 replanning instants, so consecutive action chunks continue one another rather than restarting the motion.

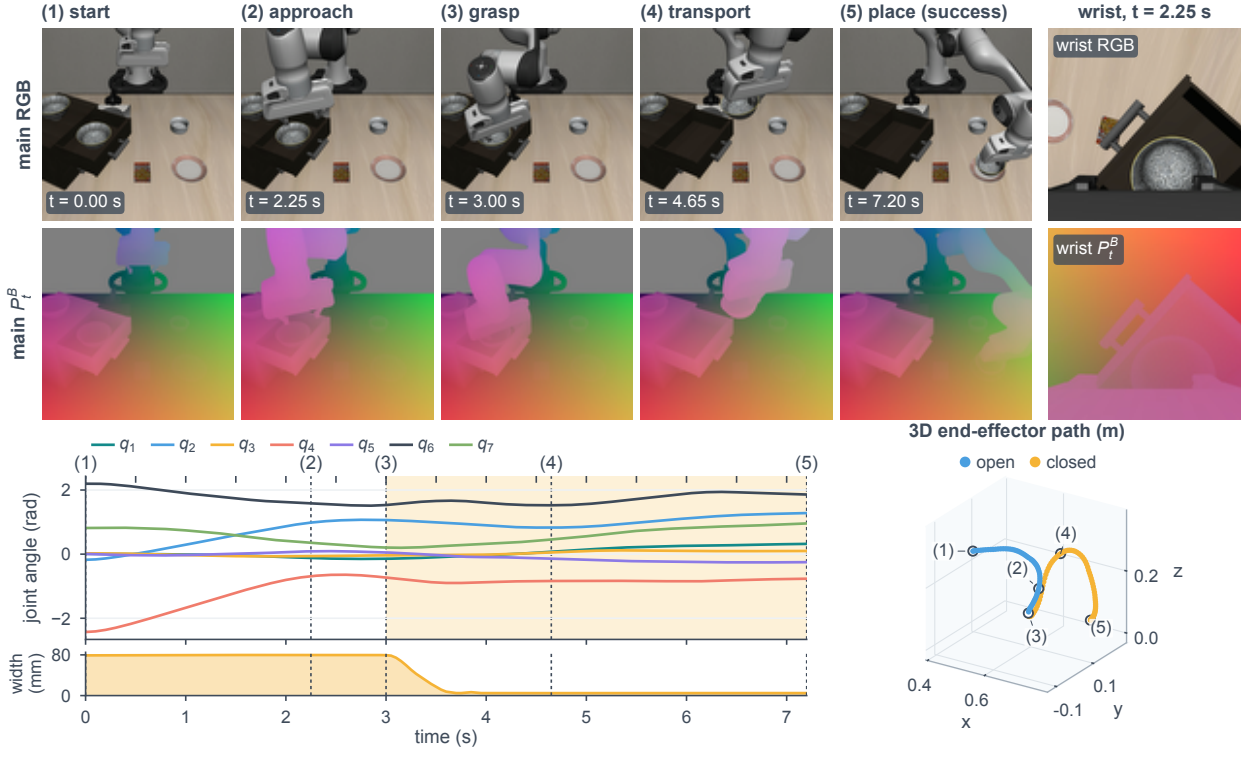


Figure 5: LIBERO-Spatial case (task 4, 144 control steps): main-camera RGB/XYZ keyframes, a wrist view, joint angles and gripper opening, and the 3D end-effector trajectory. XYZ colors encode robot-base coordinates.

5.5.2. *Case S2: camera viewpoint shift*

The LIBERO-Plus case (Fig. 6) moves the main camera backward, so the robot and the objects occupy a smaller part of the image and the table edges become visible (the inset shows the nominal view). Because the pointmap encodes coordinates in the robot base frame, static surfaces keep their colors under this change even though their positions and sizes in the image change. The executed motion closely follows the nominal case: the grasp occurs at step 50 ($t = 2.5$ s),

Table 4
Success rate (%) on LIBERO-Occ [16]. Views lists the available camera inputs; Avg. is the mean over the four suites.

Method	Views	Spatial	Goal	Object	LIBERO-10	Avg.
UniVLA [27]	Main	61.00	67.40	78.80	21.20	57.10
OpenVLA [12]	Main	39.20	37.40	65.60	20.40	40.65
OpenVLA-OFT [10]	Main	48.00	44.80	75.60	23.40	47.95
π_0 [1]	Main	52.40	56.00	72.40	16.40	49.30
$\pi_{0.5}$ [21]	Main	39.00	40.20	64.00	19.00	40.55
Fast-WAM [29]	Main + wrist	74.4	73.3	67.3	69.8	71.2
GeoAlign-WAM (ours)	Main + wrist	84.3	85.2	79.3	83.7	83.1

the transport reaches $z = 0.32$ m, and the episode ends at $t = 6.55$ s with the end effector at $(0.73,\ 0.24,\ 0.04)$ m, about 2 cm from the nominal end point. The wrist view, which the perturbation does not affect, shows the bowl in the open drawer below the gripper shortly before the grasp.

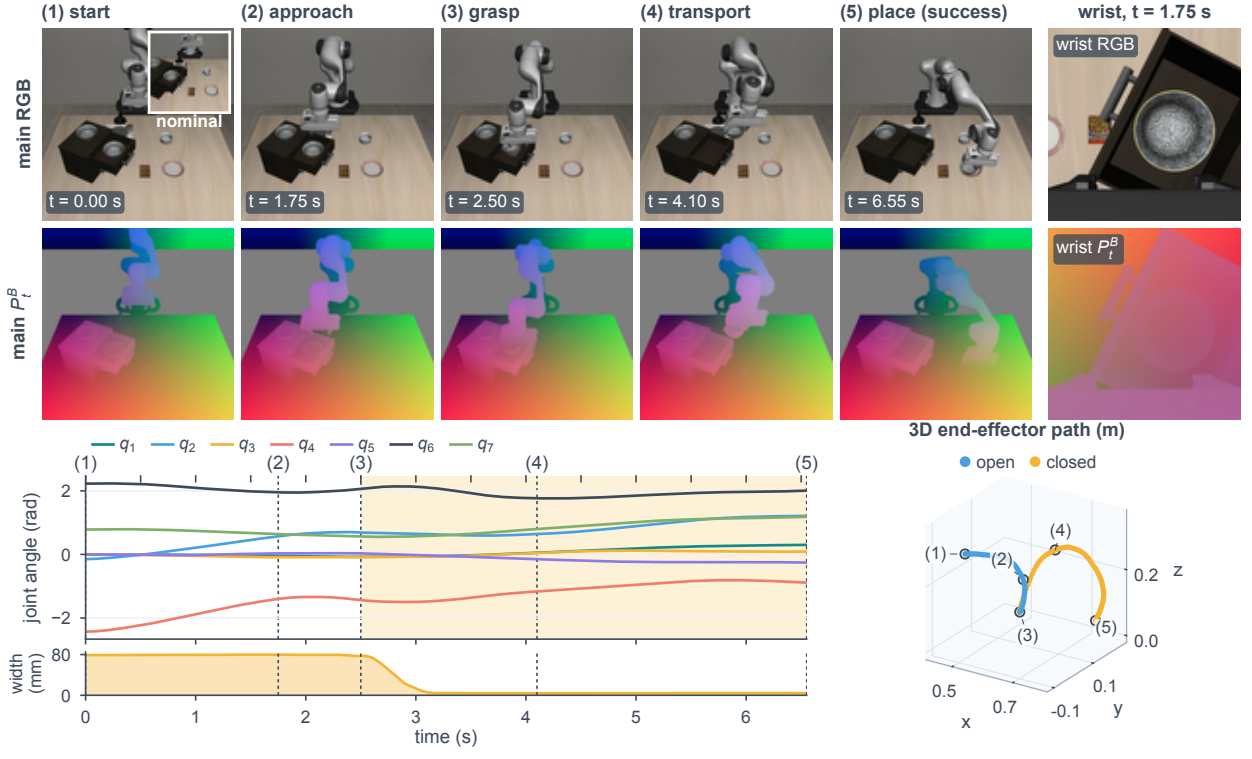


Figure 6: LIBERO-Plus viewpoint perturbation (131 control steps). The inset shows the nominal view; the remaining plots follow Fig. 5.

5.5.3. Case S3: occluded target

In the LIBERO-Occ case (Fig. 7), a wooden board stands between the main camera and the plate, leaving only part of the plate rim visible in RGB. In the pointmap, the board appears as a near-camera surface with coordinates that differ clearly from those of the plate behind it. During the approach, the wrist camera looks down on the drawer and the bowl; the board appears only at the upper left of its image and does not hide the bowl. The policy grasps at step 50 ($t = 2.5$ s), transports the bowl at a peak height of 0.35 m, about 2–3 cm higher than in the other two cases, and lowers it behind the board onto the plate; success is reached at $t = 6.5$ s with the end effector at $(0.71,\ 0.24,\ 0.04)$ m. Across the three cases, the end points differ by at most 3 cm in the base frame although the images differ substantially.

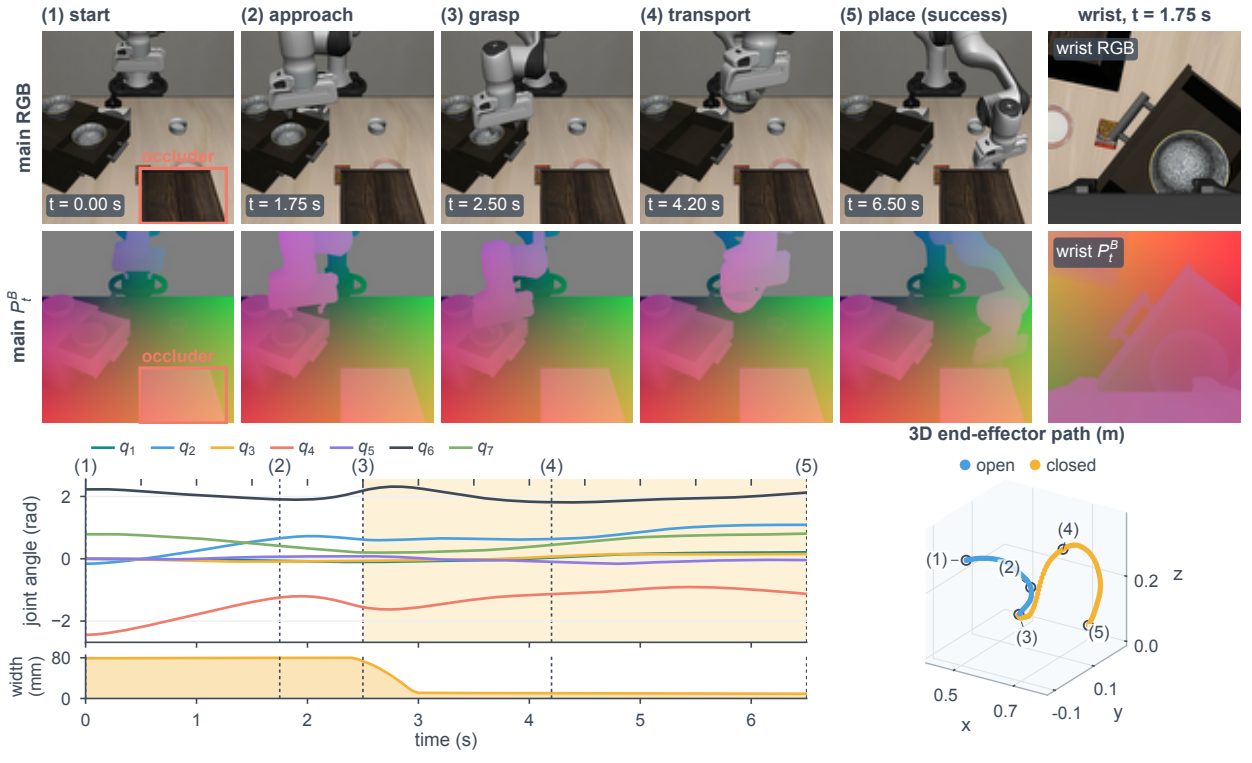


Figure 7: LIBERO-Occ case (130 control steps). The occluder is outlined in the first keyframe; the remaining plots follow Fig. 5.

6. Real-robot experiments

6.1. Setup

The real-robot experiments use a 7-DoF Franka arm with a parallel-jaw gripper (Fig. 8). Two RGB-D cameras observe the scene: an external main camera placed about 1.0 m in front of the robot base and 0.6 m above it, facing the robot, and a wrist camera mounted on the gripper. Both stream 640×480 color and depth images at 15 Hz. Intrinsics are taken from the camera drivers, the pose of the main camera in the base frame is calibrated, and the wrist camera pose follows from the hand–camera calibration and the measured robot state. Pointmaps are computed from each depth image with Eq. (8) and normalized as in simulation; pixels without valid depth are marked invalid. The policy receives the same packed 224×896 frame as in simulation.

A policy server returns 32-step chunks of seven-dimensional end-effector increments, which the robot executes in a 20 Hz control loop, requesting a new chunk before the current one is used up. Each normalized action step is scaled to at most 12 mm of translation and 0.036 rad of rotation per axis, and the controller enforces limits on contact force and on the total displacement of an episode. An operator ends each episode once the task is completed or has failed, with a hard limit of 120 s.

6.2. Tasks, data and baselines

The three tasks test placement precision, relative positioning of two objects and color-conditioned goal selection. In *Pick Cup*, the robot grasps a pink cup and places it upright on a yellow plate. In *Stack the Cup*, it grasps a yellow cup and nests it inside a green cup. In *Color Match*, it places a yellow cup on the plate of the same color while a white plate acts as a distractor. A separate policy is trained for each task from demonstrations, 50 for Pick Cup, 200 for Stack the Cup and 50 for Color Match. The baselines are:

- $\pi_{0.5}$ [21], fine-tuned on the task demonstrations.

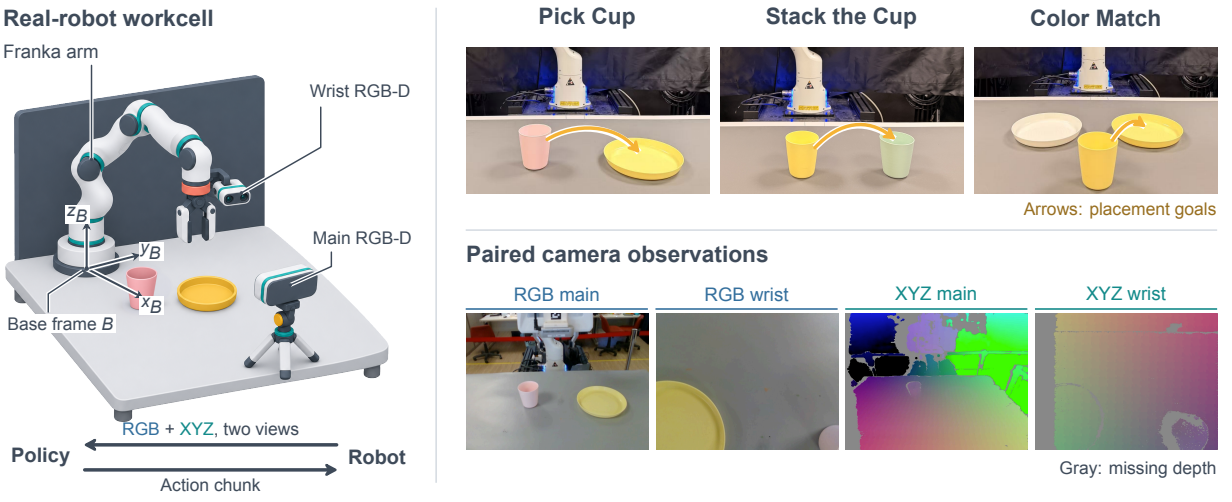


Figure 8: Real-robot workcell, the three manipulation tasks, and paired RGB/XYZ observations from the main and wrist cameras. Gray pointmap pixels indicate missing depth.

- Fast-WAM [29], using RGB only, with the same task data, backbone, action interface, and inference settings as GeoAlign-WAM.
- Diffusion Policy (DP) [5], a diffusion-based action policy evaluated on Pick Cup and Stack the Cup.

Author check. The demonstration counts are inferred from the checkpoint names (pickcup50, stack200, colormatch50). Confirm them, the training steps of each policy, the fine-tuning recipe of $\pi_{0.5}$, and the architecture, observations, and training data used for DP.

6.3. Evaluation protocol

Each reported policy–task result is evaluated over 25 trials. DP has no reported Color Match result. A trial succeeds if the final state is reached and stays stable after the gripper has opened: the cup stands upright on the plate (Pick Cup), the yellow cup rests inside the upright green cup (Stack the Cup), or the yellow cup stands upright on the yellow plate (Color Match). Outcomes are labeled by the operator at the end of each trial.

Author check. Confirm the randomization of the initial object positions and the success criteria.

6.4. Results

Table 5 reports 80%, 84%, and 88% success for GeoAlign-WAM on Pick Cup, Stack the Cup, and Color Match, respectively, corresponding to 20, 21, and 22 successes out of 25 trials. The mean is 84.0%, compared with 70.7% for RGB-only Fast-WAM and 62.7% for $\pi_{0.5}$. These are increases of 13.3 and 21.3 percentage points. Against Fast-WAM, the task-level gains are 8, 16, and 16 points, so the improvement extends across all three tasks.

The task breakdown also identifies a limit of the comparison: DP reaches 84% on Pick Cup, one successful trial more than GeoAlign-WAM, while GeoAlign-WAM leads DP by 12 points on Stack the Cup. A three-task average is unavailable for DP because Color Match was not reported. The rates are descriptive estimates from 25 trials per task; the selected rollouts below illustrate observed behaviors but do not determine the frequency or cause of failure. In particular, the results alone do not isolate the effect of alignment from geometry input or controller settings.

Table 5
Real-robot success rate (%), with 25 trials per reported task. Average is the mean across all three tasks; a dash indicates an unavailable result.

Method	Pick Cup	Stack the Cup	Color Match	Average
$\pi_{0.5}$ [21]	64	48	76	62.7
DP [5]	**84**	72	–	–
Fast-WAM, RGB only [29]	72	68	72	70.7
GeoAlign-WAM (ours)	80	**84**	**88**	**84.0**

6.5. Case studies

Figures 9–11 show one complete successful trial of GeoAlign-WAM for each task, together with a failed trial of $\pi_{0.5}$ and RGB-only Fast-WAM on the same task. The trials of GeoAlign-WAM were selected for visibility from the successful runs; the baseline trials are recorded failures, chosen because the failure is clearly visible in the main camera. The joint angles were recovered from the recorded transforms between consecutive links of the arm; forward kinematics of the recovered angles reproduces the measured end-effector position with a median error below 0.02 mm in all three trials. Times are given from the start of each recording.

The three trials share a common joint-space pattern. The approach is produced mainly by flexing the elbow (q_4) and the wrist (q_6), or the shoulder (q_2) in Color Match; the arm is still while the gripper closes and while it opens; and the transfer is carried mostly by the base and upper-arm joints (q_1, q_3).

6.5.1. Case R1: Pick Cup

The policy starts acting at $t = 7.4$ s and lowers the gripper to the pink cup, changing q_4 by -0.39 rad and q_6 by $+0.33$ rad on the way down (Fig. 9). The fingers close on the cup wall between $t = 17.0$ and 18.9 s. The cup is lifted and carried to the plate through rotations of q_1, q_3 and q_7 of about 0.2–0.3 rad each, and it is released over the plate between $t = 24.9$ and 26.1 s; it lands upright and stays in place for the last 8 s of the recording. Shortly before the grasp, the wrist camera sees the rim of the cup at the right edge of its image. Its pointmap, computed in the depth camera's own image plane, resolves the table, while the region closest to the camera returns no valid depth. Both baseline failures are spatial: $\pi_{0.5}$ grasps and lifts the cup, but the cup misses the plate and lands on the table behind it, and the RGB-only Fast-WAM approaches the cup but moves to the plate without it.

6.5.2. Case R2: Stack the Cup

In this task, the yellow cup must be lowered into a green cup whose opening is only slightly larger (Fig. 10). The gripper closes on the yellow cup between $t = 13.6$ and 15.5 s. The transfer and descent then take about 13 s, during which the arm moves slowly, with net changes of only 0.16 rad in q_1 and 0.14 rad in q_3. The cup is released inside the green cup between $t = 28.2$ and 29.4 s, and the stacked cups remain upright for the final 9 s. $\pi_{0.5}$ brings the yellow cup over the green cup, but the green cup tilts during contact and both cups fall; the RGB-only Fast-WAM releases the yellow cup beside the green cup, so that the two cups end side by side.

6.5.3. Case R3: Color Match

Here, the yellow cup must go to the yellow plate and not to the white one (Fig. 11). The gripper closes on the cup between $t = 10.9$ and 13.0 s. The transfer is the largest motion of the three tasks, with q_4 and q_2 changing by -0.52 and -0.48 rad, and the cup is carried over the yellow plate at $t = 20.0$ s, released at low height between $t = 20.6$ and 22.0 s, and settles upright on the plate at $t = 22.5$ s. In this run, the controller held the arm during the 38 control steps in which the gripper was closing and executed a 28-step lift after the grasp; these steps are logged as controller overrides, and all other motion comes from the policy. With $\pi_{0.5}$, the standing cup tips over after the gripper touches it, so the yellow plate stays empty; the RGB-only Fast-WAM lifts the cup but stops between the two plates, leaving it outside both.

Author check. State whether the gripper-closing hold and the post-grasp lift were also active in the baseline trials of Color Match.

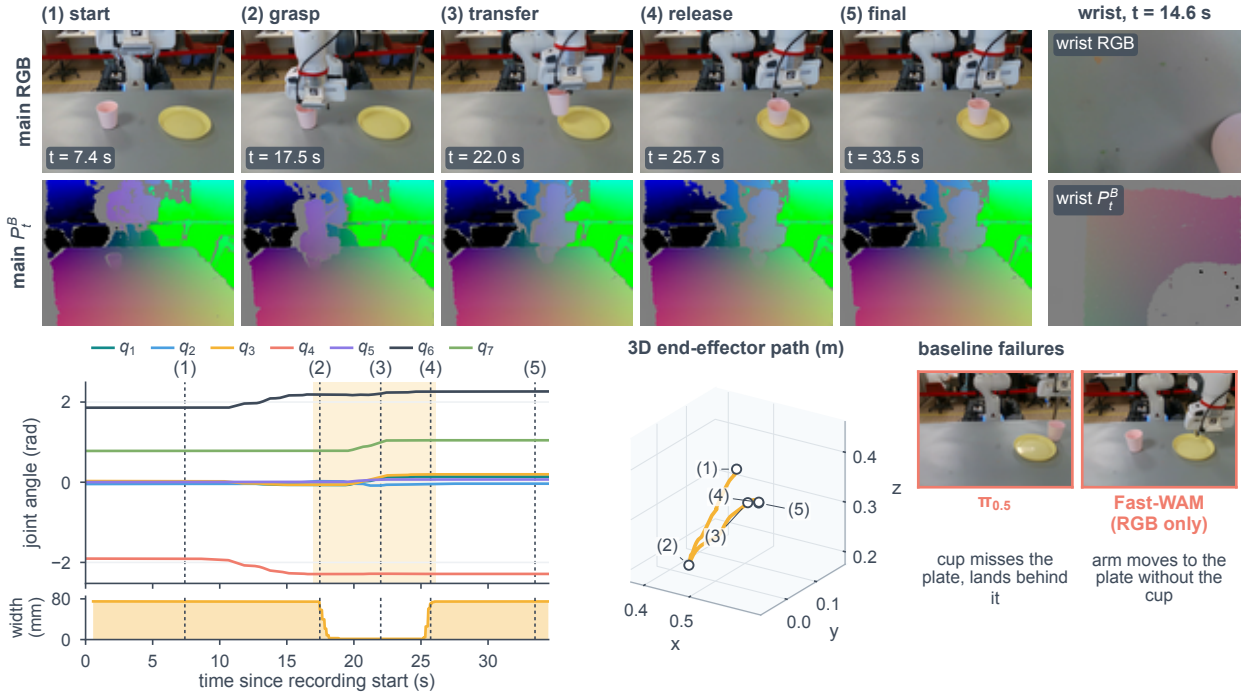


Figure 9: Pick Cup case (34.5 s recording): main and wrist RGB/XYZ views, reconstructed joint angles and measured gripper opening, the 3D end-effector trajectory, and same-task failure frames from $\pi_{0.5}$ and RGB-only Fast-WAM.

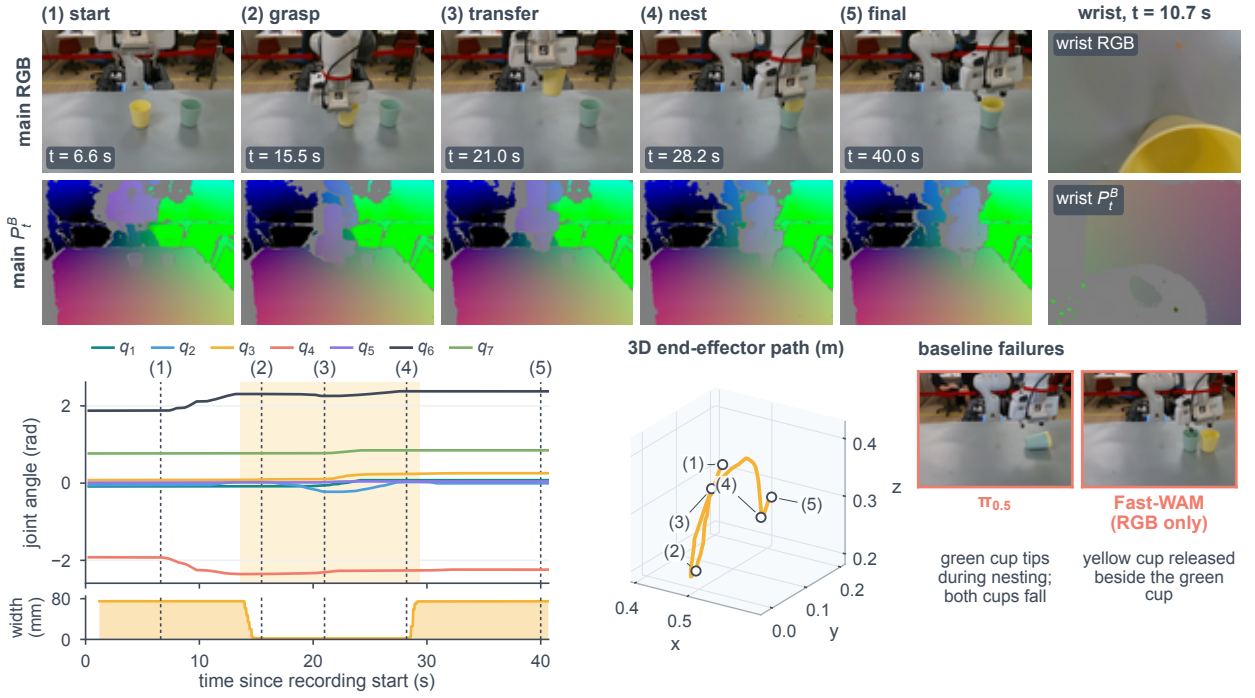


Figure 10: Stack the Cup case (40.7 s recording), with the same views and plots as Fig. 9.

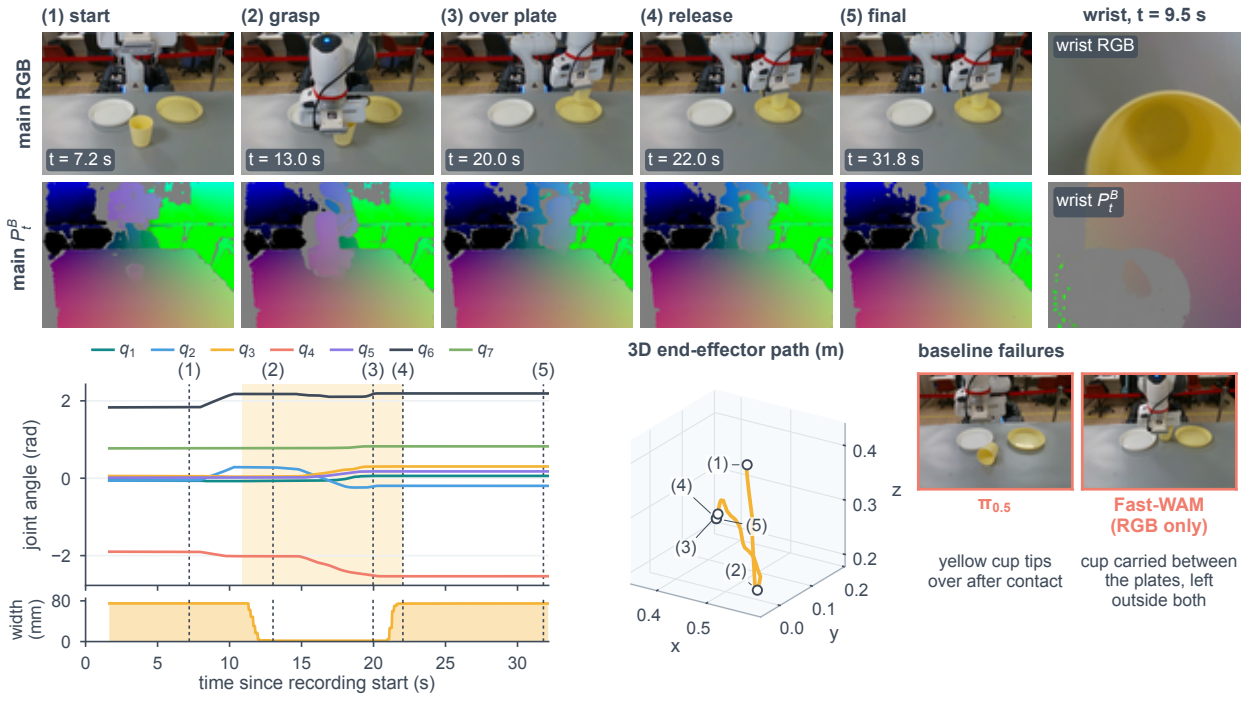


Figure 11: Color Match case (32.2 s recording), with the same views and plots as Fig. 9.

7. Conclusion

This paper presented GeoAlign-WAM, a world action model that couples a pretrained video prior with metric pointmaps in the robot base frame. RGB and XYZ observations are encoded into compatible latent grids and fused along the width dimension, a world backbone learns to predict how both evolve, and an action expert produces executable action chunks from the current context alone. A convolutional readout of the predicted future latent provides an auxiliary action chunk, and a symmetric alignment loss ties it to the expert's prediction, so that future geometric supervision shapes the policy without being required at deployment. The evaluation separates nominal performance on LIBERO from robustness to perturbations on LIBERO-Plus and to occlusion on LIBERO-Occ, and transfers the comparison to three real-robot tasks. GeoAlign-WAM achieves 97.4% mean success on nominal LIBERO, 83.1% on LIBERO-Occ, and 84.0% across the three real-robot tasks. The latter two exceed Fast-WAM by 11.9 and 13.3 percentage points, respectively; the LIBERO-Plus breakdown shows large camera-shift gains alongside unresolved weaknesses under some language and lighting changes.

The approach relies on calibrated metric depth. Near-range depth dropout of the wrist camera, visible in the real-robot close-ups, removes geometry exactly when the gripper approaches an object, and calibration errors propagate directly into the pointmaps. The alignment loss enforces agreement between two predictions, which can coincide while both being wrong, and its benefit therefore rests on the supervised world and action objectives. The fixed workspace normalization must also be set for each robot and workspace.

Future work will investigate learned metric geometry, transfer across tasks and embodiments, and uncertainty-weighted alignment. Active observation offers a further direction: context-dependent exploration and relocation in SOR-Nav [9], together with viewpoint-supported memory in SafeVantage [14], motivate deciding when additional views are needed before manipulation. Another direction is to combine geometric world features with learning from mixed-quality deployment experience, following the predictive chunk-level evaluation of PACL [22]. These extensions would require separate evaluation of observation costs and policy improvement.

A. RGB–pointmap fusion formulations

This appendix specifies the five fusion layouts considered in Section 4.4, following the modality-fusion design space of 4DNeX [4]. Width fusion is the construction used by GeoAlign-WAM. The remaining layouts and interaction operators below define alternatives for controlled comparisons; their mathematical specification does not imply that an experiment has been completed.

A.1. Paired latent grids and correspondence

Include the minibatch dimension explicitly and write the normalized modality latents as

$$X = Z^{\mathrm{rgb}}, \quad Y = Z^{\mathrm{xyz}}, \qquad X, Y \in \mathbb{R}^{N\times T_z\times h\times w\times c}. \tag{22}$$

Here N is the number of paired samples. The width w already includes the fixed main/wrist camera layout. The entries $X_{n,k,i,j,:}$ and $Y_{n,k,i,j,:}$ refer to the same sample, time and image-grid location. They need not have equal feature values: the two VAE applications encode appearance and normalized metric coordinates, respectively. We use zero-based indices below and denote concatenation along axis a by $[\,\cdot\,;\,\cdot\,]_a$.

A.2. Axis-wise fusion layouts

A.2.1. Channel fusion

At each location, concatenate the two feature vectors:

$$Z^{\mathrm{ch}} = [X;Y]_c \in \mathbb{R}^{N\times T_z\times h\times w\times 2c}, \qquad Z^{\mathrm{ch}}_{n,k,i,j,:} = [X_{n,k,i,j,:}; Y_{n,k,i,j,:}]. \tag{23}$$

A patch projection jointly mixes the paired channels. For a patch containing q grid locations, its input dimension changes from qc to $2qc$; an input projection $W^{\mathrm{ch}}_{\mathrm{in}} \in \mathbb{R}^{2qc\times d}$ produces the backbone width d. A corresponding output projection must predict $2qc$ values, which are split into RGB and XYZ velocities. The token count stays unchanged for a fixed patch size, but the input and output projections change. Channel concatenation therefore cannot be substituted into a pretrained input projection without adapting its dimensions.

A.2.2. Batch layout

Stack the modalities as separate batch entries:

$$Z^{\mathrm{ba}} = [X;Y]_N \in \mathbb{R}^{2N\times T_z\times h\times w\times c}, \qquad Z^{\mathrm{ba}}_{n,:,:,:,:} = X_n, \quad Z^{\mathrm{ba}}_{N+n,:,:,:,:} = Y_n. \tag{24}$$

Ordinary transformer attention acts independently on each batch entry. Thus, for a shared sample-wise network f_θ,

$$f_\theta(Z^{\mathrm{ba}}) = [f_\theta(X); f_\theta(Y)]_N. \tag{25}$$

Parameter sharing alone provides no within-sample exchange between RGB and XYZ. A batch-based comparison must explicitly specify how the paired outputs are reunited and supplied to the action branch; an added cross-modal attention or feature-combination module is a separate operation.

A.2.3. Frame fusion

Pack complete modality sequences along the latent-time axis:

$$Z^{\mathrm{fr}} = [X;Y]_{T_z} \in \mathbb{R}^{N\times 2T_z\times h\times w\times c}, \qquad Z^{\mathrm{fr}}_{n,k,:,:,:} = X_{n,k}, \quad Z^{\mathrm{fr}}_{n,T_z+k,:,:,:} = Y_{n,k}. \tag{26}$$

Packed positions k and $T_z + k$ share the same physical timestamp; the latter does not denote a later scene. Interleaving is a permutation of this layout: $Z^{\mathrm{int}}_{n,2k} = X_{n,k}$ and $Z^{\mathrm{int}}_{n,2k+1} = Y_{n,k}$. Such permutations need consistent modality and position labels. Temporal patches must not straddle a modality boundary unless that mixed patch is explicitly intended.

A.2.4. Height fusion

Stack the two grids vertically:

$$Z^{\mathrm{ht}} = [X;Y]_h \in \mathbb{R}^{N\times T_z\times 2h\times w\times c}, \qquad Z^{\mathrm{ht}}_{n,k,i,j,:} = X_{n,k,i,j,:}, \quad Z^{\mathrm{ht}}_{n,k,h+i,j,:} = Y_{n,k,i,j,:}. \tag{27}$$

The packed row $h + i$ corresponds to the same physical image row as i. The stacking boundary is a modality boundary, not a spatial adjacency in the scene.

A.2.5. Width fusion

The adopted construction places XYZ beside RGB:

$$\begin{aligned} Z^{\mathrm{wd}} &= [X;Y]_w \in \mathbb{R}^{N\times T_z\times h\times 2w\times c}, \\ Z^{\mathrm{wd}}_{n,k,i,j,:} &= X_{n,k,i,j,:}, \qquad Z^{\mathrm{wd}}_{n,k,i,w+j,:} = Y_{n,k,i,j,:}. \end{aligned} \tag{28}$$

This is Eq. (12) with its batch axis restored. With two views per modality, the order is RGB main, RGB wrist, XYZ main, XYZ wrist. Width fusion preserves the latent channel dimension. Its inverse is a fixed split:

$$X = Z^{\mathrm{wd}}_{:,:,:,0:w,:}, \qquad Y = Z^{\mathrm{wd}}_{:,:,:,w:2w,:}. \tag{29}$$

Each original axis length is assumed divisible by its patch size, so that a patch does not cross an artificial modality boundary. The same packing and unpacking apply to clean context, noisy future latents, and predicted future velocities.

A.3. Tokenization, positions, and attention

Let a modality grid contain $L = (T_z/p_t)(h/p_h)(w/p_w)$ patches of size (p_t, p_h, p_w), with $q = p_t p_h p_w$. Denote patch extraction and flattening by $\mathcal{P}$. For layouts that retain separate modality patches, one explicit token construction is

$$H^m_r = \mathcal{P}(Z^m)_r W_{\mathrm{in}} + e_m + e_{\mathrm{pos}}(\gamma_r), \qquad m \in \{\mathrm{rgb}, \mathrm{xyz}\}, \tag{30}$$

where $W_{\mathrm{in}} \in \mathbb{R}^{qc\times d}$, e_m is a modality embedding, and γ_r records physical time, view and image-grid position. A rotary implementation applies the corresponding indices to queries and keys instead of adding e_{pos}. Equation (30) specifies a correspondence-preserving option; it does not resolve the positional-encoding implementation check in Section 4.6. Packing alone neither supplies a modality embedding nor determines the rotary indices.

Channel fusion yields L joint tokens per sample. Frame, height and width fusion each yield $2L$ tokens per sample under the divisibility assumption, whereas batch layout yields two independent sets of L tokens. For unrestricted self-attention, the corresponding score matrices have sizes $L\times L$, $2L\times 2L$, and two separate $L\times L$ blocks, respectively. These sizes describe attention arithmetic, not measured latency or total model cost.

For joint token processing, let $H = [H^{\mathrm{rgb}}; H^{\mathrm{xyz}}]_L$. Cross-modal exchange arises through the off-diagonal blocks of

$$\begin{aligned} Q &= HW_Q, \quad K = HW_K, \quad V = HW_V, \\ S &= \mathrm{softmax}\Big(QK^\top/\sqrt{d_h} + M\Big) = \begin{bmatrix} S_{rr} & S_{rp} \\ S_{pr} & S_{pp} \end{bmatrix}, \\ H'_{\mathrm{rgb}} &= S_{rr}V_{\mathrm{rgb}} + S_{rp}V_{\mathrm{xyz}}, \qquad H'_{\mathrm{xyz}} = S_{pr}V_{\mathrm{rgb}} + S_{pp}V_{\mathrm{xyz}}. \end{aligned} \tag{31}$$

The displayed outputs are for a single attention head and lie in $\mathbb{R}^{L\times d_h}$; multiple heads are concatenated and output-projected in the usual way. Here d_h is the attention-head width. The modality blocks in this equation are distinct from the current/future/action groups in Eq. (13): RGB and XYZ can interact inside C or F, while A still cannot attend to F. A modality-packing permutation must carry the group mask and positional metadata with it.

A.4. Feature-combination alternatives

For completeness, aligned token features $U = H^{\mathrm{rgb}}$ and $V = H^{\mathrm{xyz}}$ can also be combined without packing them into one longer sequence. The following formulas define alternative interaction modules, not additional modules in the reported width-fusion model.

A.4.1. Projected concatenation and addition

For projection matrices of compatible size,

$$H_{\mathrm{cat}} = [U;V]_d W_c + b_c, \qquad H_{\mathrm{add}} = UW_r + VW_p + b, \tag{32}$$

where $W_c \in \mathbb{R}^{2d\times d}$ and $W_r, W_p \in \mathbb{R}^{d\times d}$. Both produce one token per aligned location. A single linear projection of concatenated features is algebraically equivalent to projected addition when $W_c = [W_r^\top \; W_p^\top]^\top$ and the biases match. They become different designs only when their parameter constraints or nonlinear processing differ.

A.4.2. Gated fusion

A data-dependent gate can interpolate the projected features:

$$G = \sigma([U; V]_d W_g + b_g), \qquad H_{\text{gate}} = G \odot (UW_r) + (1 - G) \odot (VW_p). \tag{33}$$

Here $W_g \in \mathbb{R}^{2d \times d}$ and $b_g \in \mathbb{R}^d$. The gate has one entry per token and output channel; σ is the sigmoid. A learned gate is not automatically a calibrated measure of depth confidence. Invalid-depth masks or confidence inputs, if used, must be specified separately.

A.4.3. Bidirectional cross-attention

Separate modality streams can exchange information through

$$\begin{aligned} \overline{U} &= U + \text{Attn}(UW_Q^r, VW_K^p, VW_V^p)W_O^r, \\ \overline{V} &= V + \text{Attn}(VW_Q^p, UW_K^r, UW_V^r)W_O^p. \end{aligned} \tag{34}$$

Both updates use the incoming U, V. A subsequent operator such as Eq. (32) supplies the combined context to the policy. Such a design introduces additional projections and attention operations, which must be included in any parameter, memory and runtime comparison.

A.5. Future readout and comparison protocol

In the formulation of Eq. (15), the convolutional readout first applies Eq. (29) to the predicted clean future, then concatenates the corresponding features along channels. This separates the transformer's modality layout from the CNN's local image neighborhoods. Changing the transformer fusion does not by itself require changing the normalized action space or the alignment target.

A controlled fusion comparison should keep the paired demonstrations, normalization, current/future visibility, action horizon and evaluation protocol fixed, while reporting changes to projections, interaction layers, positional encoding, trainable parameters and computational cost. The two modality losses must be evaluated after the appropriate inverse layout. No fusion ranking is inferred here; the experimental results remain reserved in Section 5.4.

B. Offline reconstruction of predicted future pointmaps

B.1. Scope and coordinate convention

We adopt the post-optimization procedure of 4DNeX [4] for offline reconstruction of predicted future pointmaps. It is applied after future-world generation and decoding. It is separate from the latent flow-matching and geometry–action alignment objectives, and is not part of the deployed action policy.

For a camera view, let $\widehat{\widetilde{P}}_k$ be the decoded normalized XYZ map at future time $t + \delta_k$. Inverting latent normalization before decoding and then restoring the workspace transform gives

$$\widehat{P}_k^B = \widehat{\widetilde{P}}_k \odot \boldsymbol{s} + \boldsymbol{b}. \tag{35}$$

The fitting target is expressed in the robot base frame. The procedure must retain this coordinate system and scale; independently centering the predicted frames would remove information required to interpret their positions.

B.2. Camera and depth fitting

Let $\mathbf{K}$ denote intrinsics for one camera stream, $\mathbf{R}_{C,k}^B \in \text{SO}(3)$ its camera-to-base rotation, $\boldsymbol{c}_k^B \in \mathbb{R}^3$ its camera origin in the base frame, and $d_k(u) > 0$ a depth variable at pixel u. The point represented by these variables is

$$\widetilde{P}_k^B(u) = \mathbf{R}_{C,k}^B \left[d_k(u) \mathbf{K}^{-1} \widetilde{u} \right] + \boldsymbol{c}_k^B, \qquad \widetilde{u} = (u_x, u_y, 1)^\top. \tag{36}$$

Using camera-to-base transforms makes this expression consistent with Eq. (8); it is equivalent to the inverse-extrinsic convention in 4DNeX. Its camera/depth fitting objective is

$$\underset{\mathbf{K}, \{\mathbf{R}_{C,k}^B, \boldsymbol{c}_k^B, d_k\}_{k=1}^T}{\text{minimize}} \quad \frac{1}{\sum_k |\Omega_k|} \sum_{k=1}^{T} \sum_{u \in \Omega_k} \left\| \widetilde{P}_k^B(u) - \widehat{P}_k^B(u) \right\|_2^2, \tag{37}$$

where Ω_k denotes the pixels retained for fitting. With every pixel included, this is the unweighted point-fitting objective used in the released 4DNeX implementation, up to a constant normalization. The residual is a three-dimensional distance, rather than an image-plane pixel error. The predicted pointmaps remain fixed while the camera and depth variables are fitted. RGB supplies appearance for the reconstructed points and does not enter this objective.

The released implementation uses intrinsics shared across the frames of a stream, per-frame poses and depths, a six-dimensional rotation parameterization, and Adam optimization. Different physical cameras require separate intrinsic matrices. Once fitted, Eq. (36) produces the reconstructed sequence. This camera-consistency fit does not enforce temporal point identities or establish that a predicted interaction is physically correct.

A calibrated robot may hold some camera parameters fixed instead of fitting them. In particular, a future wrist-camera pose requires the corresponding robot configuration and cannot be assumed available to the online policy. The adopted constraints must therefore be reported separately from the free-camera source formulation.

Author check. Offline reconstruction of predicted future pointmaps is confirmed. Specify whether intrinsics and poses are optimized or fixed in the implementation, how the two cameras are handled, the valid-pixel mask, initialization, any extra regularizers, optimizer settings, iteration count, and stopping criterion. The published 4DNeX solver defaults must not be reported as our settings without confirmation.

References

[1] Black, K., Brown, N., Driess, D., Esmail, A., Equi, M.R., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L.X., Smith, L., Tanner, J., Vuong, Q., Walling, A., Wang, H., Zhilinsky, U., 2025. π_0: A Vision-Language-Action Flow Model for General Robot Control, in: Proceedings of Robotics: Science and Systems, LosAngeles, CA, USA.

[2] Bu, Q., Yang, Y., Cai, J., Gao, S., Ren, G., Yao, M., Luo, P., Li, H., 2025. UniVLA: Learning to act anywhere with task-centric latent actions. arXiv preprint arXiv:2505.06111 Accepted to Robotics: Science and Systems (RSS) 2025.

[3] Cen, J., Yu, C., Yuan, H., Jiang, Y., Huang, S., Guo, J., Li, X., Song, Y., Luo, H., Wang, F., Zhao, D., Chen, H., 2025. WorldVLA: Towards autoregressive action world model. arXiv preprint arXiv:2506.21539 .

[4] Chen, Z., Liu, T., Zhuo, L., Ren, J., Tao, Z., Zhu, H., Hong, F., Pan, L., Liu, Z., 2025. 4DNeX: Feed-forward 4D generative modeling made easy. arXiv preprint arXiv:2508.13154 .

[5] Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B.C., Song, S., 2023. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, in: Proceedings of Robotics: Science and Systems, Daegu, Republic of Korea.

[6] Fei, S., Wang, S., Shi, J., Dai, Z., Cai, J., Qian, P., Ji, L., He, X., Zhang, S., Fei, Z., Fu, J., Gong, J., Qiu, X., 2025. LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models. arXiv preprint arXiv:2510.13626 .

[7] Guo, J., Li, Q., Li, P., Chen, Z., Sun, N., Su, Y., Wang, H., Zhang, Y., Li, X., Liu, H., 2026. Unified 4D World Action Modeling from Video Priors with Asynchronous Denoising. arXiv preprint arXiv:2604.26694 .

[8] Hung, C.Y., Sun, Q., Hong, P., Zadeh, A., Li, C., Tan, U.X., Majumder, N., Poria, S., 2025. NORA: A small open-sourced generalist vision language action model for embodied tasks. arXiv preprint arXiv:2504.19854 .

[9] Ji, Y., Li, Z., Cai, Y., Wu, S., Han, B.S., Lv, C., 2026. SOR-Nav: Search or Relocate? Context-Gated Exploration and Cross-Region Relocation for Object Navigation. arXiv preprint arXiv:2609.34707 .

[10] Kim, M.J., Finn, C., Liang, P., 2025a. Fine-tuning vision-language-action models: Optimizing speed and success. arXiv preprint arXiv:2502.19645 Accepted to Robotics: Science and Systems (RSS) 2025.

[11] Kim, M.J., Gao, Y., Lin, T.Y., Lin, Y.C., Ge, Y., Lam, G., Liang, P., Song, S., Liu, M.Y., Finn, C., Gu, J., 2026. Cosmos Policy: Fine-tuning video models for visuomotor control and planning. arXiv preprint arXiv:2601.16163 Accepted to the International Conference on Learning Representations (ICLR) 2026.

[12] Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E.P., Sanketi, P.R., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., Finn, C., 2025b. Openvla: An open-source vision-language-action model, in: Agrawal, P., Kroemer, O., Burgard, W. (Eds.), Proceedings of The 8th Conference on Robot Learning, PMLR. pp. 2679–2713.

[13] Levine, S., Finn, C., Darrell, T., Abbeel, P., 2016. End-to-end training of deep visuomotor policies. Journal of Machine Learning Research 17, 1–40.

[14] Lewis, S.H., Guo, Z., Chen, B., Li, Z., Lin, H., Huang, H., 2026. SafeVantage: Vantage-Aware Memory for Reliable Embodied Decisions. arXiv preprint arXiv:2609.36906 .

[15] Li, F., Song, W., Zhao, H., Wang, J., Ding, P., Wang, D., Zeng, L., Li, H., 2025. Spatial Forcing: Implicit Spatial Representation Alignment for Vision-language-action Model. arXiv preprint arXiv:2510.12276 .

[16] Li, T., Zhang, J., Wang, S., Huang, X., Wei, Z., 2026a. LIBERO-Occ: Evaluating and Improving Vision-Language-Action Models under Scene-Induced Occlusion via Viewpoint Imagination. arXiv preprint arXiv:2606.10862 .

[17] Li, Y., Wei, X., Cao, J., Wang, H., Chi, X., Bai, C., Sun, Q., Li, J., Zhang, X., Jia, P., Tang, J., Han, S., Zhang, S., 2026b. WAM4D: Fast 4D World Action Model via Spatial Register Tokens. arXiv preprint arXiv:2606.14048 .

[18] Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., Le, M., 2022. Flow matching for generative modeling. arXiv preprint arXiv:2210.02747 .

[19] Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., Stone, P., 2023. Libero: Benchmarking knowledge transfer for lifelong robot learning, in: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (Eds.), Advances in Neural Information Processing Systems, Curran Associates, Inc.. pp. 44776–44791.

[20] Pertsch, K., Stachowicz, K., Ichter, B., Driess, D., Nair, S., Vuong, Q., Mees, O., Finn, C., Levine, S., 2025. FAST: Efficient action tokenization for vision-language-action models. arXiv preprint arXiv:2501.09747 Accepted to Robotics: Science and Systems (RSS) 2025.

[21] Physical Intelligence, Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., et al., 2025. $\pi_{0.5}$: a vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054 Accepted to the Conference on Robot Learning (CoRL) 2025.

[22] Ren, Y., Yan, Y., Li, Z., Guo, J., Zeng, D., Tao, J., Yu, L., Tian, X., Lv, C., 2026. Learning from Mixed-Quality Deployment Experience for Robot Manipulation. arXiv preprint arXiv:2609.29000 .

[23] Shridhar, M., Manuelli, L., Fox, D., 2023. Perceiver-actor: A multi-task transformer for robotic manipulation, in: Liu, K., Kulic, D., Ichnowski, J. (Eds.), Proceedings of The 6th Conference on Robot Learning, PMLR. pp. 785–799.

[24] Sun, G., Du, T., Feng, K., Luo, C., Ding, X., Shen, Z., Wang, Z., He, Y., Li, A., 2026. ROCKET: Residual-oriented multi-layer alignment for spatially-aware vision-language-action models. arXiv preprint arXiv:2602.17951 Accepted to Advances in Neural Information Processing Systems (NeurIPS) 2026.

[25] Tan, S., Dou, K., Zhao, Y., Krähenbühl, P., 2025. Interactive post-training for vision-language-action models. arXiv preprint arXiv:2505.17016 .

[26] Team Wan, Wang, A., Ai, B., Wen, B., Mao, C., Xie, C.W., Chen, D., Yu, F., Zhao, H., Yang, J., et al., 2025. Wan: Open and advanced large-scale video generative models. arXiv preprint arXiv:2503.20314 .

[27] Wang, Y., Li, X., Wang, W., Zhang, J., Li, Y., Chen, Y., Wang, X., Zhang, Z., 2025. Unified vision-language-action model. arXiv preprint arXiv:2506.19850 .

[28] Ye, S., Ge, Y., Zheng, K., Gao, S., Yu, S., Kurian, G., Indupuru, S., Tan, Y.L., Zhu, C., Xiang, J., Malik, A., Lee, K., Liang, W., Ranawaka, N., Gu, J., Xu, Y., Wang, G., Hu, F., Narayan, A., Bjorck, J., Wang, J., Kim, G., Niu, D., Zheng, R., Xie, Y., Wu, J., Wang, Q., Julian, R., Xu, D., Du, Y., Chebotar, Y., Reed, S., Kautz, J., Zhu, Y., Fan, L.J., Jang, J., 2026. World Action Models are Zero-shot Policies. arXiv preprint arXiv:2602.15922 .

[29] Yuan, T., Dong, Z., Liu, Y., Zhao, H., 2026. Fast-WAM: Do world action models need test-time future imagination? arXiv preprint arXiv:2603.16666 .

[30] Ze, Y., Zhang, G., Zhang, K., Hu, C., Wang, M., Xu, H., 2024. 3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations, in: Proceedings of Robotics: Science and Systems, Delft, Netherlands.

[31] Zhao, T.Z., Kumar, V., Levine, S., Finn, C., 2023. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware, in: Proceedings of Robotics: Science and Systems, Daegu, Republic of Korea.

[32] Zhu, C., Yu, R., Feng, S., Burchfiel, B., Shah, P., Gupta, A., 2025. Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets, in: Proceedings of Robotics: Science and Systems, LosAngeles, CA, USA.

[33] Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohlhart, P., Welker, S., Wahid, A., Vuong, Q., Vanhoucke, V., Tran, H., Soricut, R., Singh, A., Singh, J., Sermanet, P., Sanketi, P.R., Salazar, G., Ryoo, M.S., Reymann, K., Rao, K., Pertsch, K., Mordatch, I., Michalewski, H., Lu, Y., Levine, S., Lee, L., Lee, T.W.E., Leal, I., Kuang, Y., Kalashnikov, D., Julian, R., Joshi, N.J., Irpan, A., Ichter, B., Hsu, J., Herzog, A., Hausman, K., Gopalakrishnan, K., Fu, C., Florence, P., Finn, C., Dubey, K.A., Driess, D., Ding, T., Choromanski, K.M., Chen, X., Chebotar, Y., Carbajal, J., Brown, N., Brohan, A., Arenas, M.G., Han, K., 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control, in: Tan, J., Toussaint, M., Darvish, K. (Eds.), Proceedings of The 7th Conference on Robot Learning, PMLR. pp. 2165–2183.